

Extracting magnetic information from Kerr Microscopy images

A report submitted to the University of Manchester for the degree of

Bachelor of Science in Computer Science

in the faculty of science and engineering

Year of submission

2025

Author

Joshua Lennon

Supervisor

Prof Thomas Thomson

Department of Computer Science

Contents

List of Tables.....	4
List of Figures.....	4
List of Algorithms	5
List of Equations	5
Abbreviations.....	5
Abstract.....	6
Declaration of Originality	6
Copyright.....	6
Acknowledgements	7
1 Introduction.....	7
1.1 Motivation	7
1.2 Problem Statement.....	8
1.3 Aims and Objectives	8
1.4 Limitations of Current Solutions.....	9
1.5 Contributions of the Work	10
1.6 Report Structure	11
2 Background and Related Work.....	11
2.1 Magneto-Optical Imaging Fundamentals	11
2.1.1 Magnetic Domains.....	11
2.1.2 The Magneto Optical Kerr Effect	12
2.1.3 Kerr Microscopy.....	12
2.1.4 Faraday effect.....	12
2.2 Computer Vision Techniques	13
2.2.1 Digital image processing	13
2.2.2 Image Correction and Stabilisation Methods.....	14
2.2.3 Information Extraction from Magnetic Domain Images	15
3 Design.....	16
3.1 Development Methodology	16
3.1.1 Agile development.....	16
3.1.2 User Stories.....	16
3.2 Design Requirements	17
3.2.1 Performance Requirements	17
3.3 Pipeline Architecture Overview.....	18
3.4 Pipeline Processing Components.....	20
3.4.1 Spatial Drift Correction Module	20

3.4.2	Detector Drift Compensation Module	21
3.4.3	Faraday Effect Separation Module	24
3.4.4	Visualisation and Analysis Interface	26
3.5	Web Application architecture	27
3.5.1	Framework Selection	27
3.5.2	Backend Processing System.....	28
3.5.3	User Interface Design.....	29
3.5.4	Data Management	30
4	Implementation	31
4.1	System Overview	31
4.2	Pre-Processing	32
4.2.1	Template Matching Approach	33
4.3	Detector Drift Correction	36
4.4	Outputs and GUI.....	39
5	Testing and Evaluation.....	39
5.1	Testing overview	39
5.2	Testing	40
5.3	Results.....	41
5.3.1	Pre-Processing	41
5.3.2	Spatial Drift Correction	41
5.3.3	Detector Drift Correction	43
5.3.4	Faraday Effect Separation	43
5.3.5	ROI Analysis and Outputs.....	44
5.3.6	End-to-End Pipeline	44
5.4	Discussion	46
5.4.1	Evaluation against success criteria.....	46
5.4.2	Comparison with existing approaches	46
5.4.3	Discussion of Limitations	46
6	Conclusions and Future Work	47
	Bibliography.....	48
	Appendix A – Front End.....	51
	Appendix B – User Guide	57
	Running the Application.....	57
	Guide.....	58

List of Tables

TABLE 1.....	44
TABLE 2.....	45

List of Figures

FIGURE 1. A SCHEMATIC OF A POLARISATION MICROSCOPE IN MOKE CONFIGURATION. [5]	7
FIGURE 2 REPRESENTATION OF DIFFERENT STANDARD IMAGE PROCESSING TECHNIQUES TO ACHIEVE A) DIFFERENTIAL MAGNETIC CONTRAST IMAGE AND B) PURE MAGNETIC IMAGE WITHOUT TOPOGRAPHIC FEATURES [10].	10
FIGURE 3 THE FARADAY EFFECT PRINCIPLE DEMONSTRATING A CHANGE IN POLARISATION ANGLE DUE TO THE NATURE OF LIGHT INTERACTING WITH A TRANSPARENT SUBSTANCE IN THE PRESENCE OF A MAGNETIC FIELD. [38].....	12
FIGURE 4 A FLOWCHART DEPICTING THE WORKFLOW OF THE ENTIRE KERR MICROSCOPY IMAGE PROCESSING PIPELINE.....	19
FIGURE 5 TEMPLATE MATCHING DEMONSTRATION FROM THE FIRST FRAME $I_1(x, y)$, WHERE THE TEMPLATE $T_{x', y'}$ IS EXTRACTED AROUND THE FIXED POINT (GREEN SQUARES).	21
FIGURE 6 A PLOT OF IMAGE INDEX AGAINST THE SUM OF PIXEL INTENSITIES FOR THAT IMAGE FOR A REAL KERR IMAGE DATASET CONTAINING 160 IMAGES.	22
FIGURE 7 12 SUBSECTIONS OF THE FIRST AND LAST IMAGE ARE USED TO COMPUTE DETECTOR DRIFT ESTIMATIONS USING LINEAR REGRESSION.....	23
FIGURE 8 INITIAL DESIGN FOR HARDCODING THE CENTRE POINT OF THE TEMPLATE FOR TEMPLATE MATCHING.	26
FIGURE 9 MODULAR ARCHITECTURE DIAGRAM FOR THE ENTIRE IMAGE PROCESSING PIPELINE SHOWING HOW EACH MODULE INTERACTS WITH EACH OTHER AND HOW DATA IS HANDLED....	27
FIGURE 10 THE FINAL INTERACTIVE CUSTOM HTML IMPLEMENTATION OF FIXED-POINT SELECTION FOR TEMPLATE MATCHING IN THE WEB APPLICATION GUI.	34
FIGURE 11 A SCREENSHOT FROM THE FINAL APPLICATION GUI DURING THE LOCATION DRIFT SETTING SELECTION STEP.....	36
FIGURE 12 EFFICIENCY DRIFT SETTINGS IN GUI.....	37
FIGURE 13 A PREVIEW OF EACH DRIFT MODEL CALCULATED INDEPENDENTLY FOR EACH SUBSECTION OF THE IMAGE SERIES. THE DASHED LINE SHOWS THE FINAL MODEL AVERAGED ACROSS PARAMETERS FROM EACH OF THE 12 SUBSECTIONS.	38
FIGURE 14 THE PROGRESS BAR IMPLEMENTATION IN THE FINAL GUI DURING A RUNTIME SESSION WHEN THE USER IS AT STEP 4 OF THE PIPELINE.	39
FIGURE 15 EARLY PLOT MAPPING OBSERVED DISPLACEMENT AGAINST IMAGE INDEX IDENTIFIED BY THE INITIAL IMPLEMENTATION OF THE TEMPLATE MATCHING ALGORITHM. THERE ARE CLEAR PERFORMANCE ISSUES THAT HAVE BEEN ADDRESSED BY THE IMPROVED IMPLEMENTATION.	42
FIGURE 16 PIXEL-SUM CURVE CORRECTED FOR FARADAY.	43
FIGURE 17 FINAL HYSTERESIS GRAPH FOR DATASET 1 OF THE REAL KERR IMAGE DATASETS WITH COERCIVITY CALCULATIONS.....	44

FIGURE 18 THE FINAL WORKFLOW PROGRESS BAR SHOWING PROGRESS THROUGH THE ENTIRE PIPELINE IN THE APPLICATION GUI.....	45
--	----

List of Algorithms

ALGORITHM 1 MAIN_APP()	31
ALGORITHM 2 UPLOAD_STEP	33
ALGORITHM 3 LOCATION_DRIFT_STEP	34
ALGORITHM 4 TEMPLATE_MATCHING	35
ALGORITHM 5 EFFICIENCY_DRIFT_STEP	37

List of Equations

EQUATION 1	13
EQUATION 2	14
EQUATION 3	18
EQUATION 4	20
EQUATION 5	21
EQUATION 6	21
EQUATION 7	22
EQUATION 8	22
EQUATION 9	23
EQUATION 10	23
EQUATION 11	24
EQUATION 12	24
EQUATION 13	24
EQUATION 14	24
EQUATION 15	24
EQUATION 16	25
EQUATION 17	25
EQUATION 18	25

Abbreviations

MOKE - magneto-optical Kerr effect

KLT - Kanade-Lucas-Tomasi

RANSAC - Random Sample Consensus

ROI – Region of Interest

Abstract

This project develops an automated image processing pipeline for Kerr Microscopy to improve image analysis and extraction of magnetic data by producing a web application interface to enhance research. As a domain specific tool, generic problems of extracting small signals and correcting for drift are explored to adapt general solutions to the specific needs of magnetic domain extraction and provide a tool that addresses an important area of scientific research. The project addresses the issue of bridging a framework utilising modern image processing and building a high-performance pipeline that simultaneously lowers the barrier of entry for researchers without a computer science background to provide a tool that can make magnetic information extraction from microscopy imaging more efficient and effective.

Declaration of Originality

No part of any of the work completed or referred to in this report has been submitted in support of an application for another degree or qualification for this or any other learning institute.

Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made only in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Dissertation restriction declarations deposited in the University Library, and The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations>).

Acknowledgements

I would like to thank my supervisor Professor Thomas Thomson, who's passionate approach to weekly meetings helped me feel confident to deliver my best work in a field I have less experience with. I am grateful for the time taken to go above and beyond including demonstrating microscope operation. I would also like to sincerely thank all my family who has supported me through these challenging years. Also, to my friends and girlfriend who were essential to keeping me going.

1 Introduction

1.1 Motivation

As the demand for smaller, faster and more efficient electronic devices continues to grow, so too has interest amongst researchers in magnetic materials for applications such as data storage, sensor applications and spintronics [1]. The need for tools that can accurately visualise and quantify the magnetic behaviours of materials to sustain this development, motivates integral research towards achieving greater understanding of the properties of magnetic materials at the microscopic level. There are many powerful magnetic material imagery techniques, among which is digitally

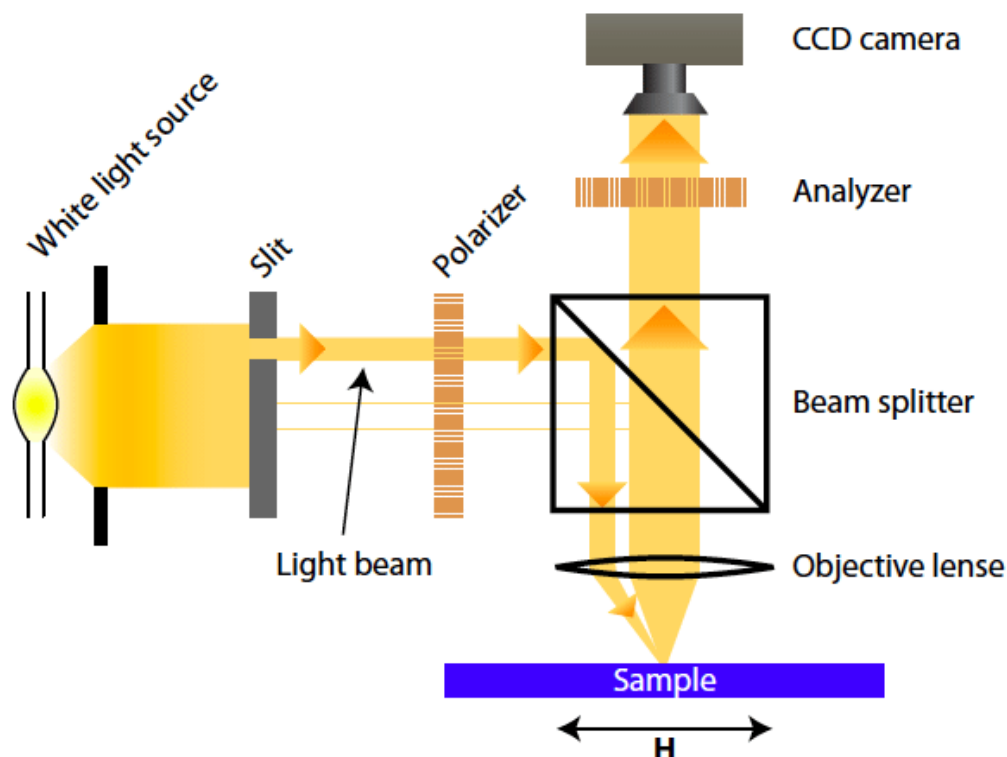


Figure 1. A schematic of a polarisation microscope in MOKE configuration. [2]

enhanced wide-field magneto-optical Kerr effect (MOKE) microscopy. MOKE enables direct magnetic domain image formation allowing visualisation of magnetisation states in materials and has emerged to become a well-established most versatile and flexible laboratory technique for the investigation of magnetic domains [3]. It does this by leveraging the way polarised light

reflects off a magnetised surface, rotating the lights' plane of polarisation – called Kerr rotation [4]. Using this effect, a series of images can be acquired with a setup like in figure 1, while sweeping an external magnetic field. The images produced can be used by researchers to observe the dynamics of domain formation, domain wall motion, and magnetisation reversal across different regions of a material [5].

The intensity of pixels within these images such as Figure 2, is dependent on the magnetic properties of that region within the material itself. Exactly how this is achieved in hand with Kerr rotation will be discussed in more detail in the background section. These images visualise magnetic domain structures and behaviour without physical contact with the sample. This non-destructive technique allows for observations of magnetisation processes under controlled conditions such as varying magnetic fields which is valuable for fundamental research [6]. The real-time nature of the observations allows for dynamic studies that complement other magnetic characterisation methods providing specific insights into surface magnetisation behaviour [7]. Consequentially there is a growing demand for enhanced magnetic data extraction methods from MOKE images, as more efficient and accurate analysis benefits material scientists developing next generation magnetic storage media, spintronic device engineers requiring precise domain behaviour characterisation and researchers in fundamental magnetism who need reliable quantitative data to validate theoretical models.

1.2 Problem Statement

MOKE is an effective and widely used technique for visualising magnetic domain behaviour, which makes meaningful data extraction from its output a crucial task. Changes in the polarisation of reflected light (Kerr rotation) are detected; however, the magnitude of this rotation is often extremely small, often around 0.003. This results in only slight contrast changes between different magnetic states in the resulting images, making the signals subtle and introducing significant discrepancy from noise and artefacts in the data. As a result, accurately isolating and quantifying these signals from raw data is a highly non-trivial task. Researchers must contend with a range of distortions including frame-to-frame spatial drift, temporal changes in detector response, and the presence of parasitic faraday effects [8] that alter the optical signal. These challenges make obtaining reliable and repeatable insights from the digital data difficult, which is why post-processing is an invaluable tool for magnetic image data extraction. This presents a fundamental challenge in digital image processing: reliably extracting and interpreting meaningful information from extremely small signal variations.

1.3 Aims and Objectives

To address the need within magnetic material characteristic research for more effective methods of data extracting from noisy MOKE imaging, this project aims to develop a software-based pipeline for accurate extraction of magnetic domain information from image sequences acquired from MOKE microscopy. The work addresses a critical gap in magnetic material characterisation research where noise, instrumental drift, and optical artifacts frequently compromise data quality. The system is designed to transform raw MOKE imaging data into quantifiable magnetic behaviour metrics through automated correction of experimental inconsistencies and sophisticated signal processing.

The objectives are outlined as follows:

- Retain spatial alignment throughout image sequence analysis and correct for fluctuations in detector sensitivity, optical pathway inconsistencies and illumination variations.
- Discriminate between true magnetic surface effects and contributions from faraday rotation in signal.
- Package analysis capabilities within intuitive graphical user interface (GUI) with clear workflow for academic reproduction of results on MOKE image data.
- Extract physically meaningful information such as coercivity, hysteresis information leveraging graphs and providing a downloadable spreadsheet of extracted data and provide a playthrough of the corrected images in series that must improve the interpretability of real experiment data.

Where possible, results are compared against expected magnetic behaviour from known samples to assess physical plausibility of outputs as well as state of the art magnetic image data processing.

1.4 Limitations of Current Solutions

The most widely used method for enhancing domain contrast in MOKE microscopy has been the subtraction of a saturated-state reference image from each frame in the image sequence [9] [10] [11]. This technique is appropriate for isolating contrast linked to magnetisation changes and can be effective in removing background features [12]. The overall effectiveness of this method is dependent on the reference mask quality which can become inconsistent with the image data across large series particularly when spatial camera drift is present in the sample [13]. An example of this subtraction can be seen in figure 2 as well as a more developed method that compensates for this by introducing a topographic reference which is multiplied against the differential image. This produces what we see in figure 2 b) giving a pure contrast image [14].

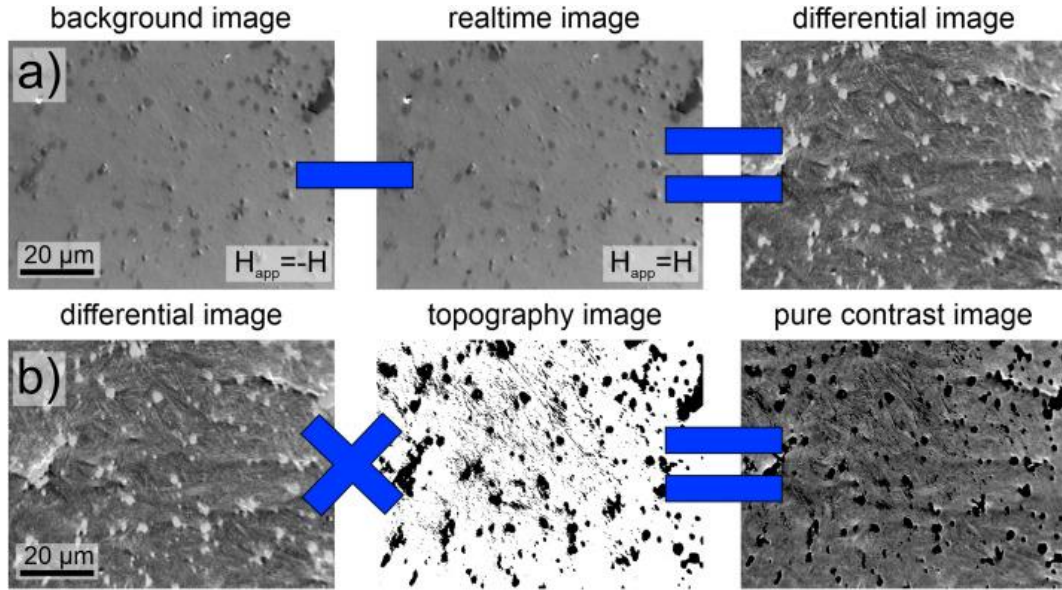


Figure 2 Representation of different standard image processing techniques to achieve a) differential magnetic contrast image and b) pure magnetic image without topographic features [10].

These methods can improve visual clarity however they remain sensitive to issues of spatial camera location drift, uneven illumination [12] and optical misalignment. On top of this, these techniques fail to address the separation of the faraday effect on surface magnetisation signals, which causes unwanted rotation of the polarised light within the microscope optics. [3] [15].

Emerging experimental techniques have seen the attempted application of deep learning towards predicting magnetic parameters from domain images produced by MOKE techniques [16]. Researchers argue their architecture establishes a foundation for comprehensive magnetic domain analysis and characterization, however these models are yet to perform significantly better than manual identification [17].

Existing solutions often exhibit a significant dependency on manual observation and lack a standardisation for reproducibility across experiments. This can raise the barrier of entry for users whose background is not reflective of the specific image processing or magnetics domain. As the Kerr rotation (the signal to be extracted) is of the order of 10^{-3} the task remains largely unsolved in magnetic data processing, there is a demand for an accessible tool that can improve on existing magnetic image data extraction techniques [18].

1.5 Contributions of the Work

To address the aforementioned limitations, this project delivers a comprehensive pipeline for post-processing magnetic microscopy image sequences, with a focus on improving the consistency and accuracy of magnetic domain data extraction through an improved image data processing application. The work tackles issues of manual dependency by using automation through user selected parameters allowing for improved reproducibility of results, as well as

correcting for issues of spatial camera location drift, uneven illumination [12] and optical misalignment. More explicitly, the core contributions of this work are:

- Development of a robust image correction pipeline that accounts for the effects of spatial location drift and detector fluctuations through spatial recognition techniques utilising template matching algorithms to correct for inter-frame motion and correct for effects that significantly compromise microscopic image data.
- Normalisation methods for the detector response for temporal fluctuations in detector sensitivity across the image series should be well developed with illumination variations and other optical pathway inconsistencies also corrected for to enable reliable, consistent comparison between frames.
- Detection and correction of parasitic Faraday effect within the magnetic signal data through decomposition algorithms allowing for isolation of authentic magnetisation changes in the image data to reveal insights into the material's magnetic characteristics.
- Introduction of a structured data export system, where users can extract domain metrics such as coercivity and magnetic hysteresis curves as well as time data, field level and other relevant experiment metrics to improve standardisation and reproducibility of results.
- Validation of produced results on real experimental data, using material samples with known benchmarks to evaluate the model's performance and accuracy. This includes comparison of perceived material behaviour from the data with expected magnetic behaviour and similar performance against image subtraction methods.

1.6 Report Structure

The rest of the report is organised as follows:

- Section 2 presents the necessary background knowledge and discusses related work.
- Section 3 is heavily focused on the foundational concepts behind the application
- Section 4 delves into the implementation of those concepts and the issues faced during this process.
- Section 5 includes testing and an interwoven deep evaluation and analysis of the results.
- Section 6 evaluates the project.

2 Background and Related Work

2.1 Magneto-Optical Imaging Fundamentals

2.1.1 Magnetic Domains

Ferromagnet substances are substances that can be permanently magnetised due to unpaired electrons [19]. These materials exhibit regions called magnetic domains where areas of uniform magnetisation have formed [20]. The boundaries between these domains, known as domain walls [21], represent transition regions where the magnetisation rotates from one direction to another. These domains are important to researchers as they are fundamental to understanding magnetic properties and behaviours. These domains influence the coercivity of materials directly [22] which is the type of magnetic data we aim to extract from the Kerr microscopy

images in this project. Manipulation and observation of domain structures is crucial for applications outlined in section such as data storage and spintronics [23].

2.1.2 The Magneto Optical Kerr Effect

The key to visualising these magnetic domains is the Kerr Effect. When linearly polarised light reflects from a magnetised surface, the polarisation plane undergoes a small rotation, known as the Kerr rotation [24]. The phenomenon occurs due to the magnetic field in the material effects how electrons interact with light causing a small rotation, called Kerr rotation.

2.1.3 Kerr Microscopy

Kerr microscopy utilises the MOKE principle to create visual representations of magnetic domain structures. A light source (typically LED) emits unpolarised light [25]. This light passes through a polariser creating linearly polarised light (the light now oscillates in one direction only as opposed to all directions of natural light) [26]. The newly polarised light reflects off the material sample and due to the previously explained Kerr effect, a rotation is observed, this rotation is different for domains with different magnetisation directions. The imaging is achieved by an analyser which converts these rotations into brightness differences, and a camera then captures the resulting digital images that we can process to observe these magnetic domain structures.

2.1.4 Faraday effect

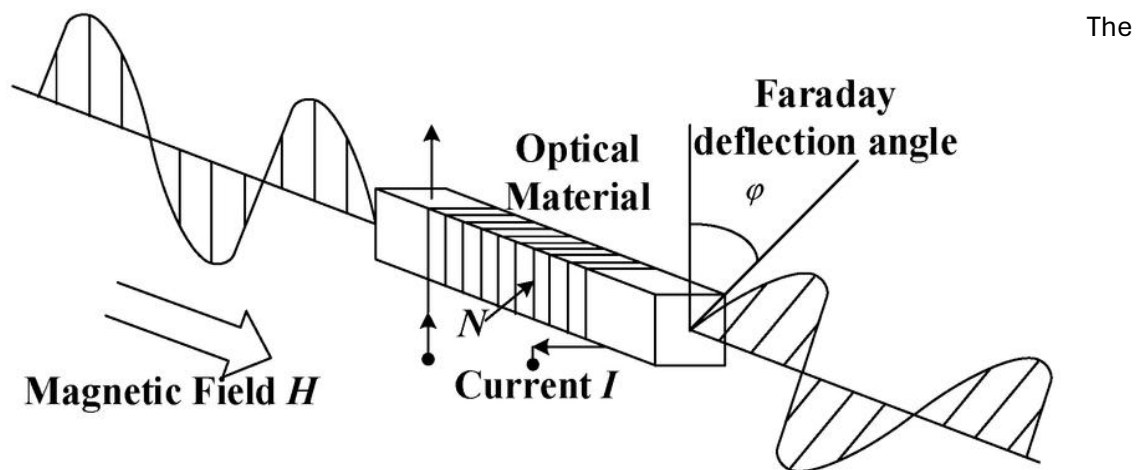


Figure 3 The Faraday effect principle demonstrating a change in polarisation angle due to the nature of light interacting with a transparent substance in the presence of a magnetic field. [38]

Faraday effect is a magneto-optical phenomenon that occurs when polarised light passes through a transparent or semi-transparent material in the presence of a magnetic field. Unlike the Kerr effect, which occurs upon reflection with the surface, the Faraday effect occurs during transmission as seen in figure 3. It's an effect that occurs in any partially or fully transparent material when a magnetic field is present i.e.: the microscope objective lens, air, mounting materials for samples etc. This effect results in an additional “Faraday rotation” within the objective lens which becomes superimposed on the Kerr rotation signal from the sample. Due to this it is a parasitic effect that affects the measured rotation without originating from the sample itself. It is an unwanted signal source as whenever a magnetic field is used to saturate

domains within the material it will also penetrate microscope components. The Faraday rotation is quantitatively described by:

Equation 1

$$\theta = V \cdot B \cdot d$$

Where:

- θ is the rotation angle.
- V is the Verdet constant (a parameter that's specific to the material).
- B common notation for field strength along the propagation direction of the light.
- d is the optical path length through the material

In Kerr microscopy the Faraday effect complicates accurate domain imaging and must be separated from the Kerr rotation signal originating from the sample itself.

2.2 Computer Vision Techniques

2.2.1 Digital image processing

The advent of digital imaging in the 1970s and subsequent development of computational analysis techniques has revolutionised the magnetic domain extraction process and more broadly image processing. Before digital technologies, researchers analysing magnetic domains relied on photographic films where information was continuously recorded through chemical processes in light-sensitive emulsions [27]. These analogue images had no discrete elements-domain boundaries and magnetic structures existed as continuous variations in optical density across the photographic medium.

Modern digital images are stored as discrete arrays containing quantities of numeric representation for its intensity or grey level (pixels). The spatial coordinates denoted by x and y of the values within the array, correspond to their position within the image [28]. For this project, The Kerr Microscopy images are digitised into 960×642 -pixel grids. Each pixel's brightness is quantised to an integer value ranging from 0 (black) to 255 (white), assigning 8 bits of information to each pixel. By assigning discrete values to the Kerr image data in this way, achieved is digital image data that can be mathematically manipulated. These digital images are represented and stored as two-dimensional matrices, where each element corresponds to a specific pixel location and contains that's pixel's intensity value:

$$I = \begin{bmatrix} I_{1,1} & I_{1,2} & \cdots & I_{1,960} \\ I_{2,1} & I_{2,2} & \cdots & I_{2,960} \\ \vdots & \vdots & \ddots & \vdots \\ I_{642,1} & I_{642,2} & \cdots & I_{642,960} \end{bmatrix}$$

Kerr microscopy images typically: exhibit low contrast (often <5% between oppositely magnetised domains [18], contain optical artifacts from the microscope and suffer from both spatial drift and drift in the efficiency of the microscope's detector. These issues require systematic and automated addressing before meaningful magnetic domain information can be extracted.

2.2.2 Image Correction and Stabilisation Methods

Spatial drift refers to when the sample shifts slightly between frames within the image series. This phenomenon can occur for various reasons and in Kerr microscopy specifically is often exacerbated by thermal expansion of components and mechanical vibrations [29]. This is essential to identify and correct when analysing domain evolution across images as it can severely compromise the ability to obtain data on the evolution of domain expansion by falsely attributing wall movement where there is none.

This is a well-documented issue across scientific imaging disciplines and has prompted the development of numerous correction algorithms. Where a fixed point within the image series is available, a fundamental approach involves identifying the features within the image sequence that should maintain fixed and using these as a reference to calculate the drift within the image. Identifying underlying camera and spatial drift information without a fixed reference would be an exceptionally untrivial task as part of noise correction, and so for this project it has been assumed that a fixed point will be made present within the sample data. For Kerr microscopy data, the correction must maintain accuracy despite the changing magnetic contrast within the images and high noise presence. Three well-established computer vision approaches were performance evaluated to determine the most suitable algorithm for magnetic domain contrast application.

2.2.2.1 Template Matching

Template matching is an algorithmic technique that has seen wide use in computer vision – where a distinct region from one frame containing the fixed point serves as a reference template across the image series. By calculating the normalised cross-correlation between this template region and each subsequent frame, pixel displacement ($\Delta x, \Delta y$) that maximises the correction value:

Equation 2

$$R(x, y) = \sum_{i,j} T(i, j) \cdot I(x + i, y + j)$$

Where T is the template, and I is the target image. Once these displacement vectors are calculated for each frame. This is a general overview of the template matching functionality, and the specific pseudocode application and mathematical explanation will be developed later in the methodology. Template matching is particularly well-suited to Kerr Microscopy imaging for several reasons:

- Good balance of computational cost which is crucial for efficient performance when working with large volumes of data like in this project despite offline nature.
- It is high performance regardless of domain changes as the nature of defining the template makes it configurable to non-magnetic features such as sample edges, fixed holes or scratches intentionally put within the sample.
- Normalised cross-correlation is a metric that provides robust matching accuracy against increase in noise and illumination changes.

2.2.2.2 Kanade-Lucas-Tomasi (KLT) Point Tracking

The Kanade-Lucas-Tomasi (KLT) algorithm is a well-documented spatial drift correction algorithm that specifically excels in identifying gradual motion within image series. It works by identifying distinct features (usually automatic feature detection using Shi-Tomasi) within an

image and tracks their movement across subsequent frames. Optical flow for locations with a significant intensity gradient in multiple directions is calculated.

KLT excels in contexts with gradual movements against noise such as particle motion, however in Kerr microscopy, magnetic domains change in appearance across the image series which could result in potential tracking errors mistaking domain boundaries as opposed to fixed identifiable features. A significant advantage of the KLT algorithm is the lack of a requirement for manual fixed-point identification which would allow full automation however with the offline processing nature of this application we are able to plan manual fixed-point identification.

2.2.2.3 Random Sample Consensus (RANSAC)-Based Affine Transformation

Random Sample Consensus (RANSAC) is a popular and robust fitting algorithm [30]. It features accurate point tracking in contexts with heavy noise presence within the data. RANSAC-based methods see standard use within visual object tracking as they are particularly effective at rejecting false correspondences between real world image data. It starts by identifying potential feature matches between frames and then randomly samples minimal sets of these matches to calculate candidate affine transformation matrices. Each candidate transformation is evaluated against all feature matches and the ones with the highest number of inliers is selected. This approach offers robustness against false patterns matches within sequential images which can be caused in magnetic image data by changing domain patterns but may struggle with the minimal to no drift present within our data as it adds complexity for primary drift translation. RANSAC is also computationally demanding and time consuming and when the target application operates in real-time, conventional approaches based on personal computers usually fail to meet the requirements [30]

2.2.2.4 Detector Drift

Detector drift refers to gradual changes in brightness or contrast within an image series due to fluctuations within either the detector's sensitivity to light (or in this case Kerr rotation) or fluctuations within the light source itself. This phenomenon can mask genuine changes in magnetic contrast and typically changes gradually due to thermal effects that evolve linearly. It is for this reason that this project assumes that the increase / decrease in detector efficiency is linear across images in series. For datasets where more complex drift patterns are suspected, our framework can be extended to incorporate higher-order models, however in practice, the linear approximation has proven sufficiently accurate for most Kerr Microscopy applications.

2.2.3 Information Extraction from Magnetic Domain Images

Another aspect of scientific image analysis involves feature extraction of text containing valuable experiment data and subsequent formatting. Optical Character recognition (OCR) converts images of text into machine encoded text that computers can process. As one of the earliest computational functionalities, its development was commercialised by IBM's OCR system in the 1950s and modernised by neural network-based methods. Automation of direct text extraction relieves the burden of researchers to manually label sample data (a large issue in modern computer vision) and eliminates manual data entry errors with standardised experiment data outputs. Extraction of field and time data allows analysis of correlation between magnetic behaviour and experimental variables across frames.

3 Design

In this chapter the design requirements for the Kerr Microscopy image processing pipeline are formalised and subsequently implemented using relevant figures and diagrams to more broadly illustrate key concepts.

3.1 Development Methodology

3.1.1 Agile development

This project used an agile-inspired development methodology. The agile framework is an iterative methodology, meaning it's product delivery begins with small primitive builds that are tested to establish functionality and are built on through repeated improvement of this testing to refine performance. Being a heavily test-driven application, it was important to use a methodology which would allow for dynamic changes in each iteration to efficiently optimise these results. This incremental approach allowed for a much faster development time to reach a minimum viable product, allowing more intention to be put into the GUI design to preserve the application's usability by the target user. The project involves building a multi-step image processing pipeline. This is well suited to agile development as it allows each component (spatial drift correction, detector drift compensation, Faraday effect separation, magnetic information extraction) to be developed, validated and improved upon independently before moving onto the next step. Furthermore, the ongoing evaluative nature of the agile development method benefited the project by allowing for pivoting to different solutions, algorithms and parameters when one either did not work or performance was not adequate.

3.1.2 User Stories

Testing was guided by user stories to inform the development cycle and ensure its alignment within the goal of creating a tool directly usable by researchers without a background in computer science. All image processing must be informed by magnetic domain identification standards and provide adequate supplement documentation for each processing objective reached. The following were the final set of stories used to drive the app development, based on discussions about the requirements of this application (expanded upon as development progressed as per the agile development pipeline):

- A researcher wants to correct microscopy images to ensure spatial camera drift within the sample is corrected.
- A physicist wants to be able to separate faraday rotation effects from Kerr effects within Kerr Microscopy data to obtain clean magnetic image data.
- A research group wants to extract quantitative hysteresis data regarding magnetic domain evolution.
- A researcher has large volumes of Kerr Microscopy data and needs an efficient way to reduce noise against magnetic domain contrast.
- A researcher wants an efficient way of visualising the evolution of magnetic domain expansion within a sample they have analysed.
- Researchers have identified possible drift within their microscope detector and want to detect this effect and correct for it within the obtained image data.
- A researcher has intentionally placed a fixed artefact within a sample and wants to be able to identify it within the image sequence as part of the spatial drift correction.

- A physicist has captured images of a sample material; however, this material makes up a small part of the image series and needs to be able to efficiently and manually identify a region of interest (ROI).
- A physicist needs a tool to identify and remove anomalous images within the image series.
- A researcher wants to correct the spatial drift within their image series where they know the drift is consistent in direction and have the option to enforce this direction.
- A physicist is aware of efficiency drift within the data however is not certain if it is consistent across the entire images and needs to have the option of correction based on the entire image or averaged subsections.
- A researcher needs to make corrections using the tool; however, they want the option to turn different steps of processing on and off.
- When determining which settings and parameters to choose to process their data, a physicist wants to be able to visualise the data and how each step will affect it.
- Researchers want a tool to automatically extract any important text information instilled within the image data directly by the microscope
- A researcher does not have a background in computer science and needs the pipeline to be intuitive and clear in its operation.

Design requirements are specified based on meeting the demands reflected in these user stories and will be referenced back to ensuring a full deliverable that meets all criteria.

3.2 Design Requirements

3.2.1 Performance Requirements

The application will need to meet a set of performance metrics that can more quantifiably measure if the system is successful. These performance benchmarks have been designed for optimal application performance for researchers analysing magnetic domain expansion:

Processing speed

The system should be able to process a typical dataset (200 frames at a resolution of 960×642) within a reasonable timeframe set at under 5 minutes.

Spatial Camera Drift Correction Stability

The standard deviation of residual drift after correction should be less than 1 pixel across frames. This ensures that the image alignment is stable making it more reliable for hysteresis graphing and visualisation.

Intensity Variation Reduction

Detector drift correction should reduce the variance of pixel intensities across frames by at least 25%, indicating improved signal stability. This minimises intensity fluctuations caused by drift in the detector making the effect of magnetic domain contrast within the material more clearly identifiable.

Faraday Effect Separation

By taking the standard deviation of the sum of the pixel values in each image and dividing it by the mean, the coefficient of variation can be calculated (*CV*):

Equation 3

$$CV = \left(\frac{\sigma}{\mu} \right) \times 100$$

Where:

- σ is the standard deviation of ROI pixel sums.
- μ is the mean of ROI pixel sums

The Faraday correction should reduce the CV indicating the Faraday effect has been removed. This can be confirmed by visually assessing the graph of pixel sums against each image and observing whether there is a flat region at the beginning and end of the data before magnetic domains begin to become saturated and the data pattern emerges.

Memory Management

The system should efficiently manage memory resources by handling large images series (up to 500 frames at full resolution) without performance suffering.

Scalability

The processing pipeline should scale efficiently with the number of frames and use programming practices that allow the application to be adapted to handle advancing image resolution demands as microscopy technology advances.

Compatibility

The application needs to be widely accessible as researchers may be running the system on a variety of browser platforms making maintaining performance across major modern browsers important.

Portability

When a session is started, all files necessary for the runtime should be stored in a temporary directory that is cleared. This makes the program self-contained from previous runs and local requirements should be more portable across systems.

Responsiveness

The UI should always remain responsive during image processing, providing user feedback that indicates when processing is taking place and what progress has been made.

3.3 Pipeline Architecture Overview

The image processing pipeline developed for this project follows a sequential workflow, where the architecture consists of self-contained modules that build upon the previous step of the analysis. Figure 4 illustrates the complete pipeline with an overview of the architecture and data flow for each stage of this development. It demonstrates how the workflow progressively works towards refining Kerr Microscopy images to refine magnetic domain information extraction.

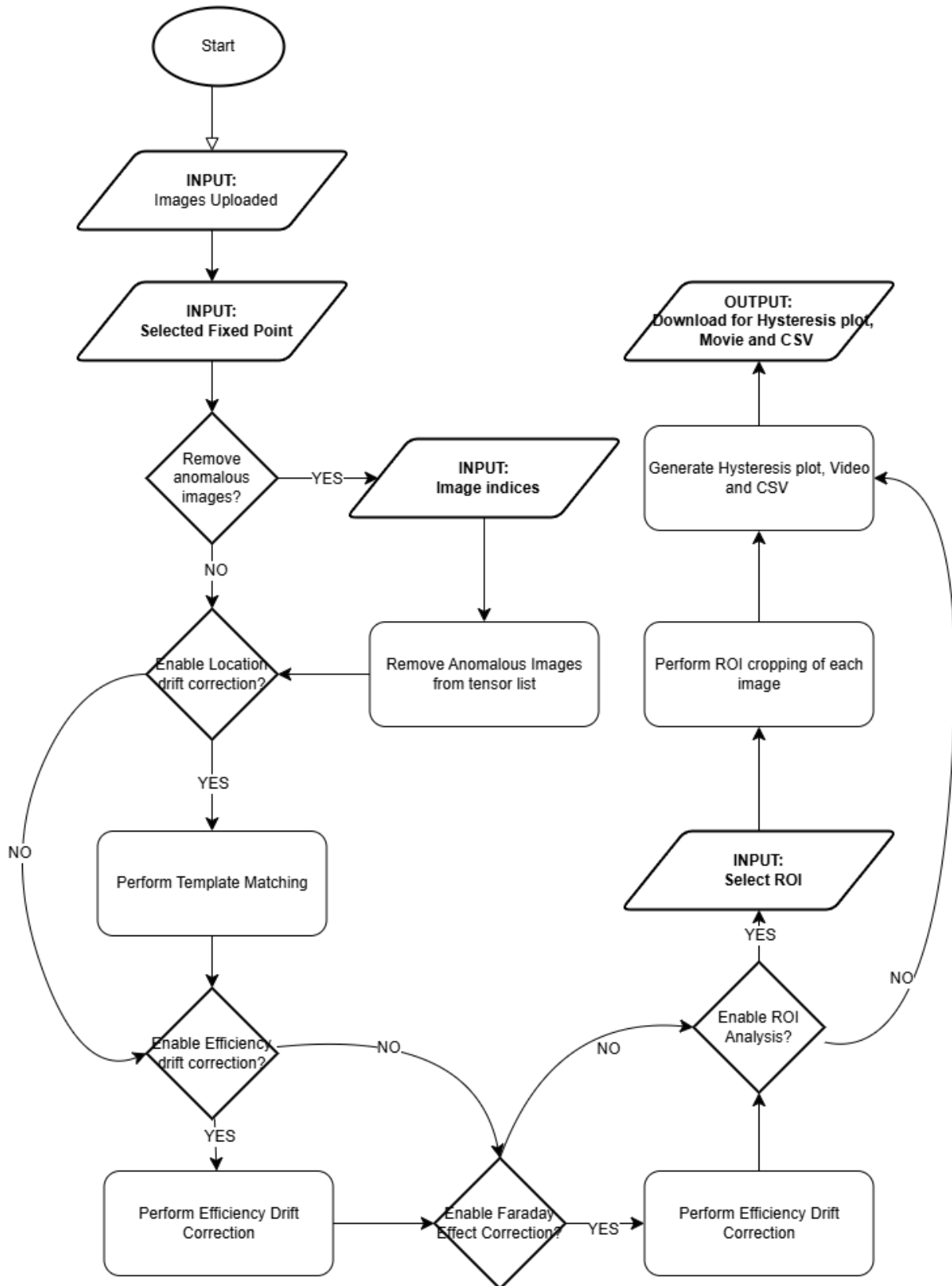


Figure 4 A flowchart depicting the workflow of the entire Kerr microscopy image processing pipeline.

The pipeline involves many operations but can be abstracted to four primary processing stages that maintain the key functions, each addressing a specific issue of noise reduction and image

analysis:

- Spatial Drift Correction
- Detector Drift Compensation
- Faraday Effect Separation
- Magnetic Domain Analysis

Each component of the system including these key steps are to be designed as a modular unit with well-defined inputs and outputs allowing for individual testing and optimisation as this is a results-based application. This design specification also allows future replacement if superior algorithms become available in line with the agile development methodology.

3.4 Pipeline Processing Components

3.4.1 Spatial Drift Correction Module

Researchers have developed a broad understanding of template matching algorithms including the Viola-Jones framework which uses Haar-like features and integral images to rapidly detect objects [31]. Recent research has shifted toward low-level feature combinations and machine learning-based techniques, which offer advantages in flexibility and speed though they require training data labels and introduce complexity [32]. Ultimately, template matching remains a foundational concept of addressing spatial drift correction, which addresses the problem of apparent microscope / sample movement between frames. This drift can cause pixel sums to be inaccurate and compromise the visualisation of magnetic domains. The correction algorithm that is used is based on template matching techniques that identify matching regions across frames as it was identified in section 2.2 to be the strongest choice for this project. This section will focus on dissecting the mathematical foundations of the algorithm, whilst pseudocode and other implementation details will be discussed in section 4.

The primary foundation for the template matching method that will be implemented is the normalised cross-correlation function, which is a method of measuring a score for the similarity between a template T and a region within the image I at position (x, y) :

Equation 4

$$R(x, y) = \frac{\sum_{x', y'} [T(x', y') \cdot I(x + x', y + y')]}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$$

Where:

- $T(x', y')$, is the template image (which will be used as the user allocated reference region).
- $I(x + x', y + y')$ is the corresponding region in the target image.
- The sum adds all template pixel coordinates (x', y') .
- $R(x, y)$ ranges from -1 (the worst possible match) to 1 (the region has been found exactly).

The value of $R(x, y)$ must be calculated for all possible (x, y) template coordinates to determine which is the best match between the template image and the corresponding region in the target image. The coordinates that maximise the value of $R(x, y)$ are identified as the location of the best match. As the fixed point is being used as the template region, if the coordinates of the

template have moved across frames, we can identify this change in (x, y) as the spatial drift expressed as $(\Delta x, \Delta y) = \arg \max_{(x, y)} R(x, y)$.

This process can be visualised in figure 5 where a template is defined from image 1 (I_1) and matched in frame $I_n(x, y)$.

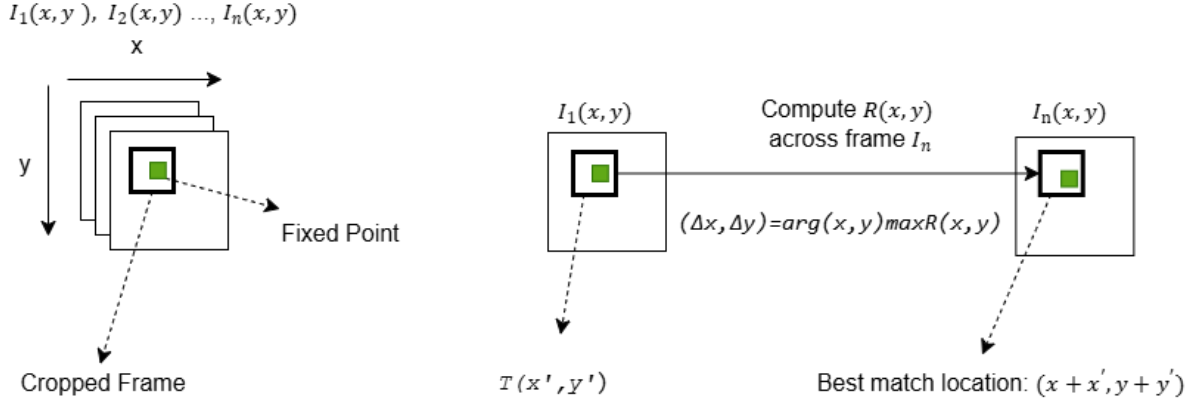


Figure 5 Template matching demonstration from the first frame $I_1(x, y)$, where the template $T(x', y')$ is extracted around the fixed point (green squares).

3.4.2 Detector Drift Compensation Module

Detector drift compensation addresses variations in overall image intensity (via pixel sums) that are not present within the data due to fluctuations in microscope detector efficiency. As discussed in section 3, the model for detector efficiency assumes a multiplicative linear effect on the recorded pixel intensity:

Equation 5

$$I_{image}(x, y, t) = E(t) \cdot I_{true}(x, y, t)$$

Where:

- $I_{image}(x, y, t)$ is the pixel intensity at time t in the image at coordinates (x, y) .
- $I_{true}(x, y, t)$ is the true pixel intensity at time t in the image at coordinates (x, y) assuming there was no drift in the detector.
- $E(t)$ is the multiplicative coefficient that creates either a linear increase or decrease in the data over time that is dependent on t time.

The goal is to estimate $E(t)$ as accurately as possible. We can analyse the sum of pixel intensities across the image sequence as $S(t)$:

Equation 6

$$S(t) = \sum_{x, y} I_{image}(x, y, t)$$

$S(t)$ can be visualised as a graph such as figure 6 for every value of t .

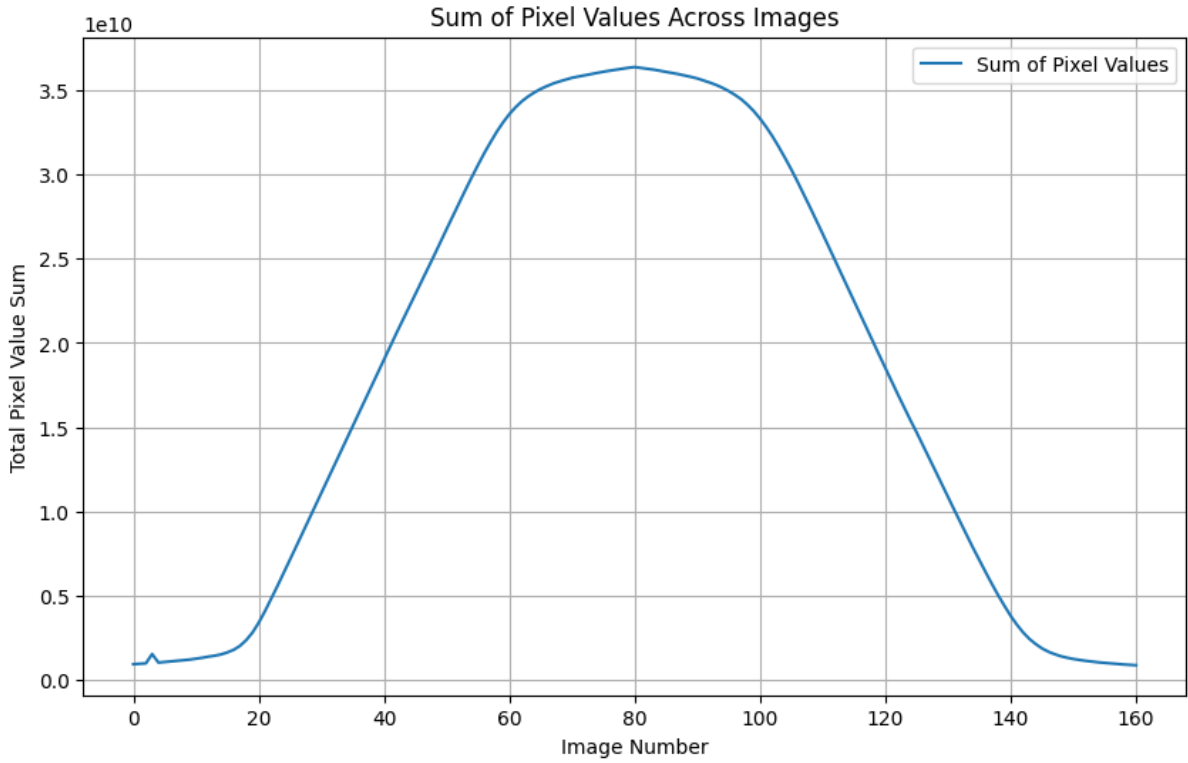


Figure 6 A plot of image index against the sum of pixel intensities for that image for a real Kerr image dataset containing 160 images.

A global increase or decrease across $S(t)$ are assumed to be primarily the result of changes in detector efficiency as explained in section 3.3 due to the difference in behaviour of regional magnetic domain evolution and global efficiency variation.

Equation 7

$$E(t) \approx \alpha t + \beta$$

Where α is the gradient (rate of the drift across the image series) and β is the intercept or initial efficiency. These parameters are estimated using linear regression on sampled points from the initial and final values of t within $S(t)$:

Equation 8

$$[\alpha, \beta] = \arg \min_{\alpha, \beta} \sum_{t \in T_{\text{sample}}} (S(t) - (\alpha t + \beta))^2$$

Where:

- T_{sample} is a set that includes the first and last few images (starting the first few from $t = 0$ and ending the last few at $t = N - 1$) to minimise the influence of magnetic domain activity.

This finds the values of $[\alpha, \beta]$ that minimise the sum of pixel intensities across the set of sample images T_{sample} which represent areas where Kerr rotation should not be present within the data. The efficiency component $E(t)$ is removed from the signal to measure how far from the actual

signal (I_{true}) the data is, then finding the values of $[\alpha, \beta]$ for $E(t)$ that reduces this distance the most. The entire component is to be squared ensuring a straight line that best fits the total pixel intensity $S(t)$ is found by making all differences positive and scaling larger errors for clarity.

This is the basis of the linear drift correction that will be implemented into the image processing pipeline. To improve the performance of this model, an option will be provided for the user to account for potential regional variations in drift within the images, by dividing the images into a grid of subsections and computing the total intensity instead for each subsection of the image. The formula $S(t)$ can be adapted:

Equation 9

$$S_k(t) = \sum_{(x,y) \in \text{subsection } k} I_{\text{image}}(x, y, t)$$

Where:

- k is the index for each subsection. In figure 7, $k \in \{1, 2, \dots, 12\}$.

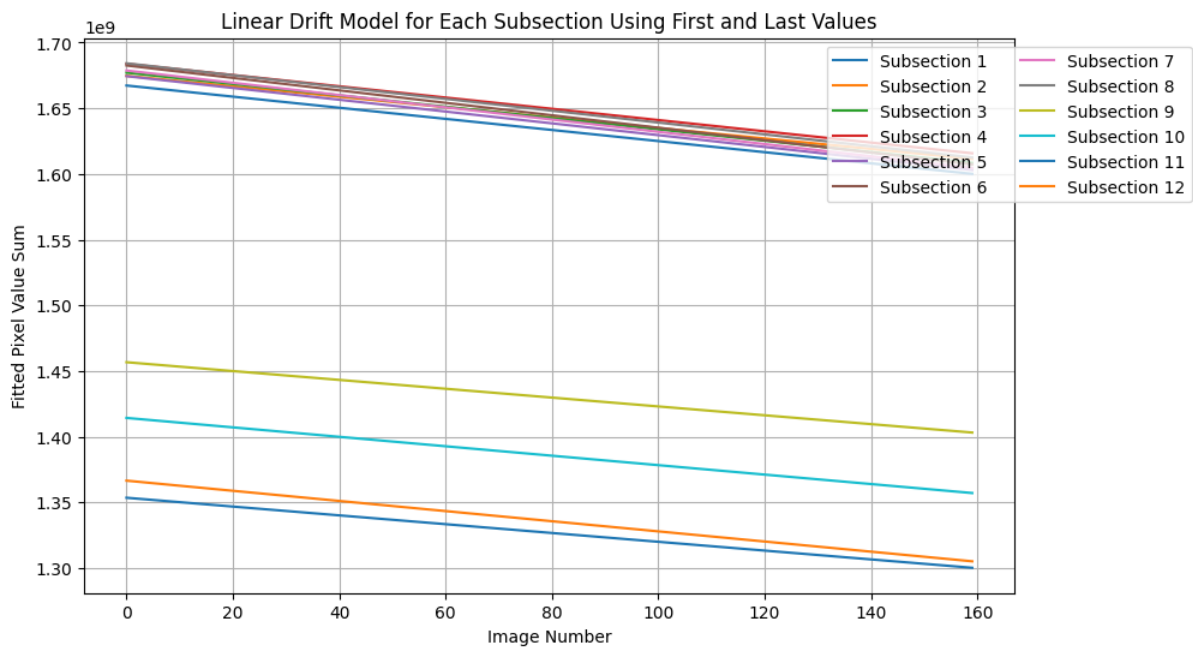


Figure 7 12 subsections of the first and last image are used to compute detector drift estimations using linear regression.

Figure 7 shows how an example estimation model across $S_k(t)$ for each of 12 subsections, can vary across subsections within an image maintaining a clear declining drift in the data. Linear regression is performed in the same way for each subsection k respectively to estimate the parameters for that specific subsection $[\alpha_k, \beta_k]$. The final drift model is obtained by averaging these parameters across each subsection to satisfy a global value for $E(t)$:

Equation 10

$$\alpha = \frac{1}{K} \sum_{k=1}^K \alpha_k, \quad \beta = \frac{1}{K} \sum_{k=1}^K \beta_k$$

Where:

- K is the total number of subsections.

Once these parameters are obtained, the correction can then be applied. Each image is normalised to the effect of the efficiency drift to produce a corrected set of images by rearranging the formula:

Equation 11

$$I_{image}(x, y, t) = E(t) \cdot I_{true}(x, y, t)$$

Equation 12

$$I_{corrected}(x, y, t) = \frac{I_{observed}(x, y, t)}{E(t)} = \frac{I_{observed}(x, y, t)}{\alpha t + \beta}$$

Equation 11 demonstrates how the obtained α, β parameters that minimise the model and comprise the drift coefficient $E(t)$ are substituted into Equation 12 to compensate this effect.

3.4.3 Faraday Effect Separation Module

The Faraday effect, which introduces a parasitic superimposed additional Kerr rotation effect on the signal from the sample, manifests as a global increase in pixel intensity across the entire image. The intensity within the image will be modelled once more, this will help to demonstrate contrast with the drift correction module and maintain clarity. This calculation assumes that the Faraday effect introduces a global intensity trend that is approximated as a component that superimposes on top of the Kerr contribution to the intensity of the image that we want to visualise:

Equation 13

$$I_{image}(x, y, t) = I_{Kerr}(x, y, t) + I_{Faraday}(t)$$

In concept this keeps things relatively simple, $I_{Kerr}(x, y, t)$ is the Kerr effect contrast at a pixel, and $I_{Faraday}(t)$ the global contribution assumes to be uniform across pixels in the image at time t .

Using this model the Faraday contribution needs to be estimated. Modelling and visualising the total intensity over time can be used as a proxy for the actual Faraday effect when Kerr effect changes are localised to (x, y) as opposed to the global effect of Faraday:

Equation 14

$$S(t) = \sum_{x,y} I_{image}(x, y, t)$$

Now $S(t)$ is obtained which represents the sum of pixel intensities at time t and can be similarly visualised in figure 6. This can be used to estimate the Faraday contribution as a piecewise linear trend within $S(t)$ that changes the influence of the magnetic field across the image sequence:

Equation 15

$$I_{Faraday}(t) \approx \alpha t + \beta$$

Where α and β are parameters that are estimated separately for the initial and final trend of $S(t)$. These parameters are estimated by fitting a linear model to the total intensity $S(t)$ for the first and last N images:

Equation 16

$$\begin{aligned} S(t) &\approx \alpha_{\text{initial}}t + \beta_{\text{initial}} \quad \text{for } t < T_{\text{mid}} \\ S(t) &\approx \alpha_{\text{final}}t + \beta_{\text{final}} \quad \text{for } t \geq T_{\text{mid}} \end{aligned}$$

Where T_{mid} is the midpoint of the sequence. The parameters are estimated using linear regression on the selected points. Another method the user can select as this method can result in a disconnect at the midpoint of the faraday trend depending on noise within the data, is to ensure a consistent Faraday effect trend model. An average gradient is calculated:

Equation 17

$$\alpha_{\text{avg}} = \frac{|\alpha_{\text{initial}}| + |\alpha_{\text{final}}|}{2}$$

The Faraday estimation trend $I_{\text{Faraday}}(t)$ is then recalculated with adjusted intercepts to this continuity within the trend of the pixel summation $S(t)$ at midpoint within the image series T_{mid} :

Equation 18

$$I_{\text{Faraday}}(t) = \begin{cases} \alpha_{\text{avg}} \cdot \text{sign}(\alpha_{\text{initial}}) \cdot t + \beta_{\text{initial}}, & \text{for } t < T_{\text{mid}} \\ \alpha_{\text{avg}} \cdot \text{sign}(\alpha_{\text{final}}) \cdot t + \beta_{\text{corrected}}, & \text{for } t \geq T_{\text{mid}} \end{cases}$$

Where:

- α_{avg} is the average of the gradients from the initial and final linear fits.
- $\text{sign}(\alpha_{\text{initial}})$ just represents the sign (-1 or +1) of the gradient's initial fit (upward or downward trend within $S(t)$). This is the same for $\text{sign}(\alpha_{\text{final}})$
- β_{initial} is the intercept from the initial segment
- $\beta_{\text{corrected}}$ is the intercept corrected to ensure continuity at the midpoint of the data.
- t continues to be the time step (or image index in this proxy) where T_{mid} is the midpoint of the image sequencing data.

This strategy effectively isolates the Kerr signal enhancing the clarity of domain structures for researchers to extract information.

3.4.4 Visualisation and Analysis Interface

Clear objectives based on the research application of this tool have been established for the interface design of this application. This section outlines more specific mock-ups of the GUI design process and how it informs the backend development. The user must be able to interactively identify the fixed point within the image series. This requires a more creative solution to interface design. Initial designs such as figure 8 relied on the user's ability to manually identify the coordinates of the fixed region on a grid and specify the centre

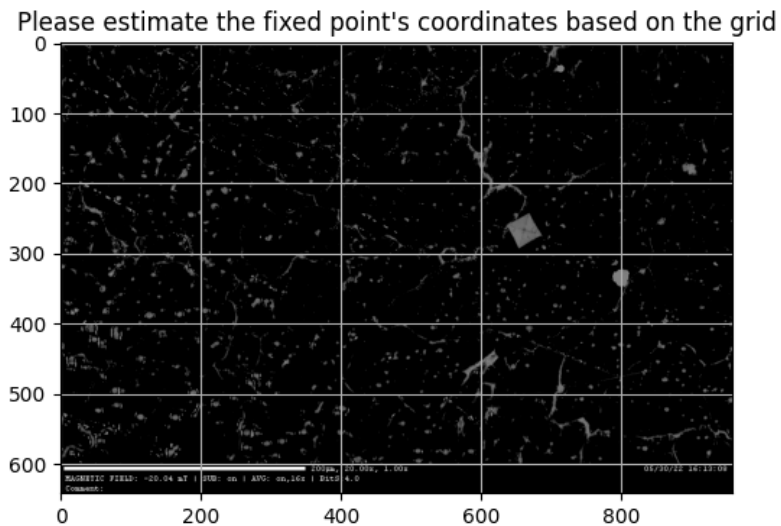


Figure 8 Initial design for hardcoding the centre point of the template for template matching.

coordinates to be used in template matching. This approach was utilised as part of the adopted agile software approach to be improved upon later iteratively once functionality within the backend was established. A mock-up of an interactive implementation can be thought of in figure 5 from section 3.5.2 (Spatial Drift Correction) depicting template matching, where the user could be imagined sliding around and resize a 'cropped frame' such that it identified the fixed point.

The pipeline should feature a workflow tracking system that features these steps with status updates. A mock-up design of this GUI implementation method is seen below:

1. **Upload** - Completed
2. **Tracking** - Completed
3. **Location Drift** - Pending
4. **Efficiency Drift** - Pending
5. **Faraday** - Pending
6. **ROI** - Pending
7. **Results** - Pending



Where:

- The blue bar on the right represents total pipeline and the green bar represents progress made through the pipeline.

Parameters for each step of this pipeline will be activated by the user primarily using checkboxes. Additionally, information boxes displaying when a process is completed and when a process has failed should be present in the demand to maintain a feedback relationship with the user. This GUI specification addresses demands from the user stories for an intuitive and interactive interface that provides feedback to researchers.

3.5 Web Application architecture

An overview of the application architecture and how each module interacts with each other can be seen in figure 9. The details of each of these modules is discussed in this section.

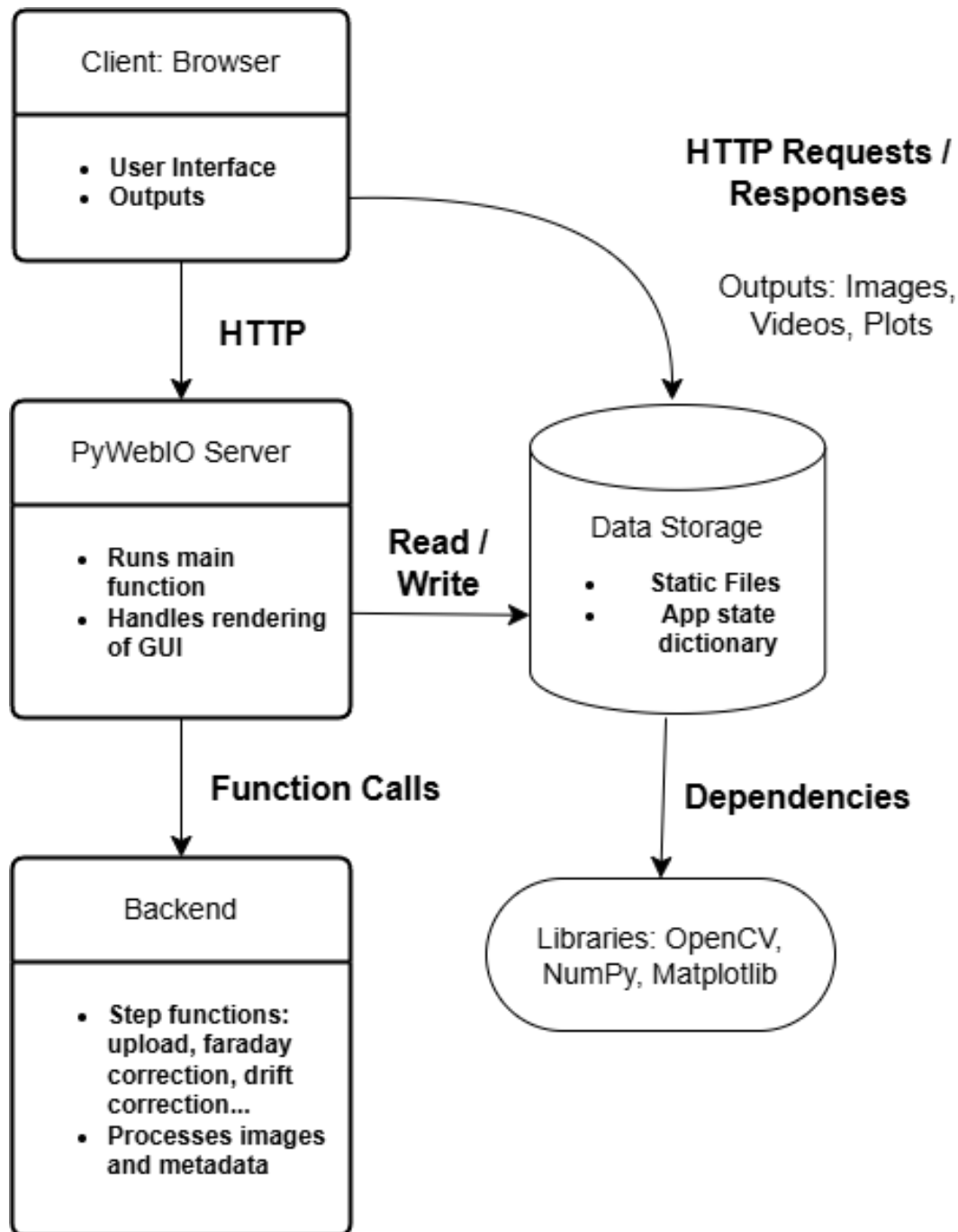


Figure 9 Modular architecture diagram for the entire image processing pipeline showing how each module interacts with each other and how data is handled.

3.5.1 Framework Selection

The Kerr Microscopy image processing pipeline is designed as a deliverable to be a web-based application. To meet this requirement multiple frameworks were considered. Streamlit, an open-source Python library, is a standard amongst Artificial Intelligence (AI) and Machine

Learning (ML) academics to deliver interactive front ends with minimal code. It is a great tool for rapid development of dashboard and user interfaces (UIs), but it lacks native support for advanced interactive image manipulation (like dragging and resizing shapes). Due to the project's objectives outlining clear academic usability, the somewhat restrictive nature this library offers could make functionality such as interactive ROI selection unnecessarily complex. Furthermore, resource constraints could negatively affect overall efficiency - which was outlined as a requirement- as well as functionality, particularly when working with large datasets which are a necessary capability for this system. For these reasons Streamlit was considered, however ultimately rejected in favour of the PyWebIO. The image processing pipeline application is implemented using the PyWebIO python library. PyWebIO enables the design of a more intuitive interactive interface for researchers while maintaining the somewhat minimised setup complexity to accelerate UI development keeping focus on refining the image processing pipeline. It does this by providing a similarly Python-centric Api to that of Streamlit, with a declarative approach that allows quick prototyping to iterate on UI components giving flexibility in line with agile software development protocol. Importantly the ability to embed custom Hypertext Markup Language (HMTL) and JavaScript allowed for the implementation of more sophisticated interactive features such as draggable shapes for user ROI specification. For these reasons it was the PyWebIO framework which was considered the stronger option for this project.

3.5.2 Backend Processing System

The backend processing system forms most of the app's development, being responsible for handling all image processing functionality outlined in the project pipeline architecture. It should be built using python and leveraging the following libraries: OpenCV, NumPy, SciPy and Matplotlib and should implement modular processing as outlined in section 3.5. The backend needs to operate as a series of sequential steps, each corresponding to a user-configurable processing module. This approach further aligns with the agile processing development approach outlined earlier in section 3. Based on discussed objectives and requirements, key features of the backend should include:

Modular Processing

There needs to be a main function responsible with executing the pipeline's distinct modules in the correct order. This can be accomplished by implementing each step as a standalone function, providing support for the agile methodology of iterative and independent testing of each section.

User-Driven Configuration of Backend Processing

The backend must support user-driven configuration, addressing user stories such as the ability to disable each step of processing and customise parameters. This is important in ensuring the pipeline is resilient to varying experimental conditions across different types of image data. This can be made possible by introducing features such as allowing the user to enforce consistent spatial drift direction and using subsection for detector efficiency correction.

OCR and Data Extraction

Pytesseract is a python library that provides access to an OCR engine. This can be used along with regular expressions to detect and parse timestamps, field levels and the scale within the image data. This fulfils the requirement outlined in the user stories for automatic text extraction.

Preprocessing (e.g., thresholding, denoising) can also be deployed to enhance the data extraction if necessary, however due to the digital unchanging nature of the text it is unlikely to enhance performance.

This backend will be hosted by initiating a PyWebIO lightweight server. The user of a server-side architecture in this way reduces the dependence for client-side installation, making the tool more portable and accessible to researchers who may be operating with a minimal setup. An installation script and a *README.txt* outlining all dependencies and pre-requisites required for running the app will be provided.

3.5.3 User Interface Design

The Front-end is responsible for providing an interface between researchers and the image processing pipeline functionality. This needs to be in the form of a web-application where the interface is simple yet informative enough to inform the user's decision making when utilising the tool. As a tool designed for specialised use within magnetic research, the UI is not the primary artefact of this deliverable however must be designed to facilitate this. The UI will provide an interactive experience using PyWebIO's output and input modules addressing the user story of creating a tool usable by researchers without a computer science background. The UI should reflect the backend, presenting the pipeline as a step-by-step workflow. It should guide users through each module of the processing pipeline including through image upload, parameter configuration, and result visualisation. Key aspect of the UI design must include:

Procedural Navigation

The UI should organise the pipeline into seven steps displayed in a navigation bar, with an accompanying progress bar that corresponds to the progress the user has currently made within the pipeline. This supports agile development by allowing users to test and validate each module independently where the final application reflects the development process.

Robust Error Handling and Feedback

The UI should provide feedback through a combination of success and error messages to inform the user when submitting settings and moving through sections. Loading indicators can help to catch errors with processing by ensuring users are aware of the applications status. When parameter settings have been left blank, this should be dealt with appropriately through error handling to ensure the application can continue to operate. The image uploading process should also prompt the user to upload images if nothing is currently uploaded.

Interactive Elements to Support User Inputs

Settings configuration should be interactive and intuitively support the users intuition where possible. Custom HTML should be used to allow users to interactively select fixed points and ROIs by dragging and resizing rectangles on a canvas. This was a requirement outlined in our user stories for manual ROI selection and fixed artifact identification.

Real-Time User Visual Feedback and Results

The user should be able to view how each parameter will affect the data throughout each step of the process. This should be achieved through a combination of intermediate results such as corrected images, displacement plots and hysteresis curves being available to the user during the pipeline configuration. This requirement addresses the user story demand of visualising the

impact of each processing step to enable researchers to make informed decisions about parameter settings.

Overall, the design should prioritise usability through real time intractability and feedback wherever possible while maintaining simplicity.

3.5.4 Data Management

The data management system is responsible for how the application handles the image data, results and processing. It needs to be designed to support large volumes of Kerr Microscopy images as per the user story for handling extensive datasets. Taking this into account the data management system must include:

Image Handling

There should be the ability to upload multiple images using PyWebIO's file uploading functions. The OpenCV library can be used to decode the images and store them in different arrays of image tensors. There should be separate arrays corresponding to the full image set, the cropped image set, and the cropped image set with anomalous images removed to ensure data integrity is maintained throughout the processing workflow. The system will allow the user to specify through a comma separated textbox which images are anomalous, which addresses the story for removing anomalous data.

State management

A dictionary to track the applications current state is necessary. Centralising the state management so that data is accessible across the pipeline at every processing step throughout the application session supports the modular pipeline structure and can be achieved by creating a single globally accessible structure that holds all the information needed for the application along with the current state of the application. The dictionary should hold both the state of the application, and the data associated with that state such that the set of images at that state of the process is stored allowing to easily omit steps of the process and access the image data reflected in that step of the pipeline.

Static File Handling

The PyWebIO framework is not designed for embedding or playing videos as it does not have built in static file serving. This issue is referred to as a lack of native static file serving and is something that needs to be addressed by this application. Processed outputs, such as MP4 videos can be saved to a static directory that is automatically created at runtime. Output files and displayable media can be written to this directory temporarily while the session is running to enable browser playback and download. This directory is reset at the start of each session which ensures robust data management.

Memory Efficiency

To accommodate the nature of magnetic domain information extraction's dependence on large datasets, the system must process images in memory without persistent storage. The application is designed for single-session use, meaning all data can be processed within memory during the session. In memory storage is sufficient for this workflow, as data doesn't need to persist at the end of a session as all necessary downloads are provided to the user. The use of this memory-based processing method means that there is a greater demand on the systems memory as outlined in the performance requirements section (3). To address this,

NumPy arrays will be leveraged to store image data as tensors as these arrays are highly efficient data structures in python image processing.

Overall, the data management system is designed to be lightweight, relying on in-memory storage and temporary file handling, which aligns with the performance requirements for efficient processing (section 3).

4 Implementation

In this chapter, the implementation of the Kerr Microscopy image processing pipeline will be looked at in detail, translating the design requirements from chapter 3 into a functional web-based application. Moreover, some of the issues faced when implementing this design will be discussed along with the method details and resolution to these issues. The sections of this chapter will follow the order that the app was developed, beginning with each step of the pipeline and resulting in the app construction.

4.1 System Overview

An overview of the final system can be found in this section, followed by a detailed plan of how this was achieved; later sections will discuss modules as they were sequentially developed. The pipeline is implemented as a modular workflow with sequential states controlled by a function *main_app* and tracked using the *app_state* dictionary. The final workflow consists of 7 steps comprised of modules that handle:

- Image uploads and OCR-based data extraction - *upload_step*
- Allowing users to interactively select a fixed point for spatial drift correction - *fixed_point_step*
- Correcting spatial drift using template matching - *location_drift_step*
- Compensating detector efficiency drift - *efficiency_drift_step*
- Separating Faraday effect contributions - *faraday_effect_step*
- Processing user-defined ROIs - *roi_analysis_step*, *process_roi*
- Generating and display results - *final_processing_step*, *display_results*

The final application uses the *app_state* dictionary to centralise data throughout the runtime and calls each step based on the state data:

Algorithm 1 *main_app()*

PROCEDURE *main_app()* :

```
BEGIN
  INITIALISE app_state = {}
  CALL upload_step(app_state)
  CALL fixed_point_step(app_state)
  CALL location_drift_step(app_state)
  CALL efficiency_drift_step(app_state)
  CALL faraday_effect_step(app_state)
  CALL roi_analysis_step(app_state)
```

```
CALL final_processing_step(app_state)

END
```

When deciding on how to implement this pipeline there were technology factors that needed to be considered. One of the first considerations with this project was critical decision of the programming language to use. This decision needed to balance handling of large microscopy datasets, and flexibility to support agile development as well as keeping the programming intuitive to be adapted later. Python emerged as the clear frontrunner due to its predominant use within scientific image processing, with an importantly large and well documented ecosystem of libraries such as OpenCV for computer vision and NumPy for numerical operations. Python's readability and simplicity also aligns with the user story of creating an intuitive tool for researchers, as its syntax is more accessible to users without a background in software development. Alternative image processing platform like MATLAB were considered, given historically prevalent use as a specialised image processing tool withing research. MATLAB however is a proprietary software, making portability (an established requirement for the project) outside of university research environments difficult. Additionally, web integration for MATLAB sees far less support than it's pythonic counterparts creating unnecessary complexity in developing the front end of the application. Finally, C++ was considered as performance-based alternative, however, was not suited to the rapid prototyping and iteration-based development style this project demanded. The initial development involved drafting each module in a Jupyter notebook, allowing independence of cells for an agile work environment that made iterative testing and development efficient. A list of planned technologies was created before programming began:

- Python 3.8: The core programming language of this project.
- PyWebIO: A python-based web framework for UI and simple web-app development.
- OpenCV: For image processing in template matching and image transformations.
- NumPy: for image operations and handling in all transformations.
- SciPy: For linear regression.
- Matplotlib: for visualising the data (important for testing development).
- Pytesseract: For OCR-based extraction of timestamps and metadata.
- Pandas: For creating data frames to handle field data and combine extracted metadata.

4.2 Pre-Processing

Development began in a Jupyter notebook prototyping algorithms on a single dataset before they would later be generalised across other sets of Kerr microscopy image data. The notebook loaded images from a fixed directory using OpenCV which evolved into the *upload_step* module that handles image uploads via PyWebIO's *file_upload* function. Images are decoded using OpenCV's *cv2.imread* in greyscale mode and stored as NumPy arrays in *app_state*. Pytesseract extracts timestamps, field data and scale from predefined image regions, however the accuracy of OCR extractions were initially low. Issues of detected metadata formatting being inconsistent resulting in processing errors were addressed by implementing regular expressions to standardise parsing. Despite the digital nature of the text extraction, parsing failures arose and were reduced through a combination of thresholding and denoising before OCR application. This processing was initially developed early in the Jupyter notebook development cycle and was later implemented as the *extract_time* function which crops the ROI, applies thresholding and extracts the text using OCR. Users can select and

upload multiple individual image files that are stored as tensors seen in Algorithm 2 *upload_step*. Anomalies were initially hardcoded, however final implementation saw them input by the user through comma separated indices into a textbox.

Algorithm 2 *upload_step*

```
PROCEDURE upload_step(app_state):  
    BEGIN  
        images ← UPLOAD_FILES()  
        FOR EACH image IN images:  
            tensor = DECODE_IMAGE(image, greyscale=True)  
            APPEND tensor TO app_state['current_images']  
            timestamp ← EXTRACT_TIME(tensor)  
            APPEND timestamp TO app_state['timestamps']  
        END FOR  
        Anomalies ← USER_INPUT('anomalies')  
        app_state['anomalies'] ← anomalies  
        field_data ← READ_CSV('field_data.txt')  
        app_state['field_data'] ← MERGE(field_data, timestamps)  
    END
```

Extracted data is stored in *app_state*.

4.2.1 Template Matching Approach

The first correction applied after preprocessing is spatial correction using template matching. This method was selected as the final method after comparing the results of three approaches in the Jupyter notebook on a single Kerr microscopy image dataset: template matching, RANSAC-based affine transformation and KLT point tracking. The module's implementation utilises OpenCV's *cv2.matchTemplate* to track a user-defined fixed point across frames.

The *fixed_point_step* allows users to interactively select a fixed point via a draggable rectangle on a canvas. As the fixed-point coordinates were hard coded during development, issues arose implementing the interactive functionality. The first iteration of the web application utilised the Streamlit framework, which does not offer native support for interactive web elements as discussed as a limitation in section 3.6.1. This was resolved in the second iteration which opted for a PyWebIO approach that allowed the use of custom HTML / JavaScript elements seen in figure [10].

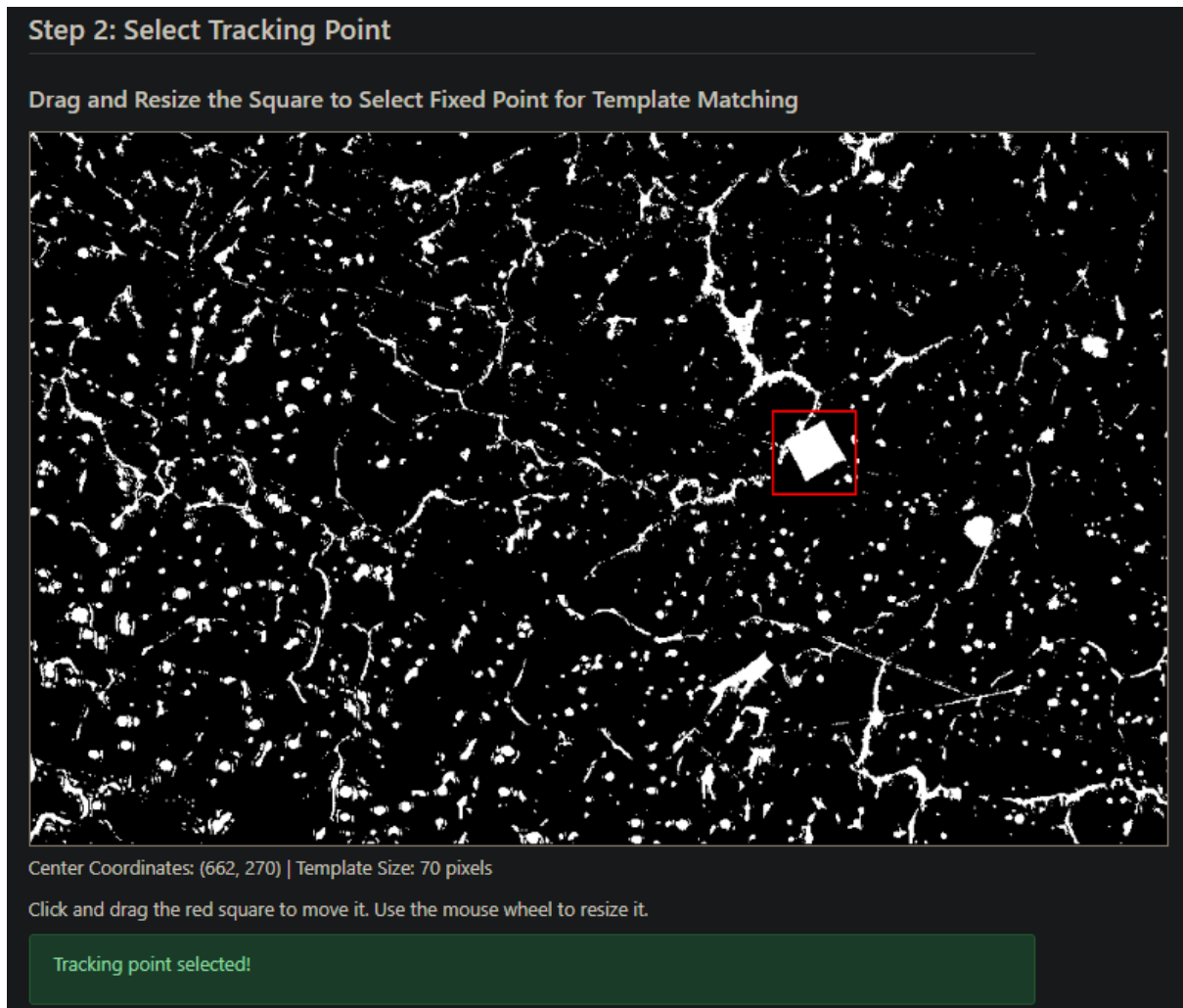


Figure 10 The final interactive custom HTML implementation of fixed-point selection for template matching in the web application GUI.

The coordinates of the centre point are stored in `app_state` and the `location_drift_step` extracts a resizable rectangular template (initially 20×20 pixels) around the coordinates that represent the template region. Normalised cross-correlation is then used as the metric for identify the fixed point (`cv2.TM_CCOEFF_NORMED`) to find the best match for the template in each image. The pseudocode for the implementation of how image data is parsed in and out of the template matching step, and a good visualisation for how the `app_state` dictionary parses data through each module can be seen below:

Algorithm 3 `location_drift_step`

```
PROCEDURE location_drift_step(app_state):
BEGIN
    x_fixed, y_fixed ← app_state['fixed_point']
    displacements = []
    FOR i FROM 1 TO LEN(app_state['current_images']):
```

```

        x, y ←
        TEMPLATE_MATCHING(app_state['current_images'][i-1],
        app_state['current_images'][i], x_fixed, y_fixed)

        APPEND (x, y) TO displacements

    END FOR

    app_state['displacements'] ← displacements

    corrected_images ← [app_state['current_images'][0]]

    FOR i FROM 1 TO LEN(app_state['current_images']):

        (dx, dy) ← displacements[i-1]

        Shifted_image ←
        SHIFT_IMAGE(app_state['current_images'][i], (dx,dy))

        APPEND shifted_image TO corrected_images

    END FOR

    app_state['current_images'] ← corrected_images

END

```

the fixed point is specified by the app state and carried into the module by the *app_state* dictionary. It goes through every image and compares it to the next by passing them to the template matching function as *image1* and *image2*:

Algorithm 4 template_matching

```

FUNCTION template_matching(image1, image2, x_fixed, y_fixed)

BEGIN

    Extract template centered at (x_fixed, y_fixed) from image1

    Normalised cross-correlation between image1 and template

    x ← x_best + (template_size / 2) - x_fixed

    y ← y_best + (template_size / 2) - y_fixed

    RETURN (x,y)

END

```

The basic implementation of template matching can be seen as a template is extracted and normalised cross-correlation is calculated to find the best match in *image2* returning *x* representing the displacement horizontally and *y* the displacement vertically across the images. Occasionally this template matching implementation would return abnormally large displacements in initial implementations. This issue was addressed in the final implementation by introducing the user-configurable parameters seen in figure 11: minimum displacement threshold (default 0.5 pixels) and maximum step size (default 10 pixels).

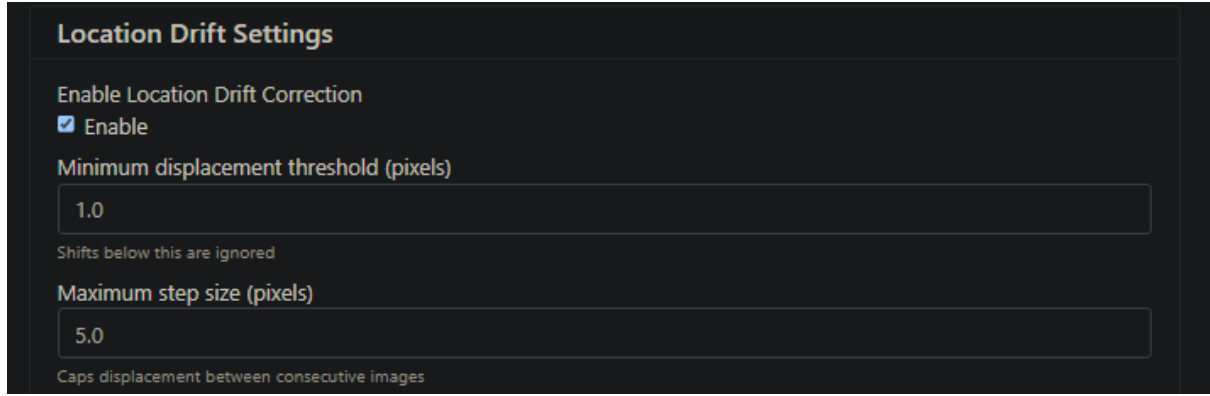


Figure 11 A screenshot from the final application GUI during the location drift setting selection step.

Large template sizes can introduce performance issues. To resolve this the template size was optimised to a maximum size of 200×200 pixels.

4.3 Detector Drift Correction

The final *efficiency_drift_step* module corrects efficiency drift within the detector causing changes in image intensity. It began by modelling the drift as described in the design in section 3:

Equation 5

$$I_{image}(x, y, t) = E(t) \cdot I_{true}(x, y, t), \quad E(t) \approx \alpha t + \beta$$

In early working models of the pipeline implementation, new matrices were calculated to preserve image data after every image transformation in the pipeline. These matrices containing intensity of each pixel are summed to obtain the total intensity of the pixels within each image $S(t) = \sum_{x,y} I_{image}(x, y, t)$. To preserve memory during runtime this was later adapted to the current images implemented in the *app_state* dictionary that also ensures processing steps skipped within the pipeline have their data integrity maintained. Linear regression is used to calculate α and β using *scipy.stats.linregress* from SciPy library on the first and last few images. Initial implementation saw the first and last 5 images used in the calculation which resulted in overfitting to noise. Different image sets were experimented with until the final implementation settled on using the first and last images for efficiency balanced with the improved performance.

This implementation was later improved upon through the implementation of subsection-based

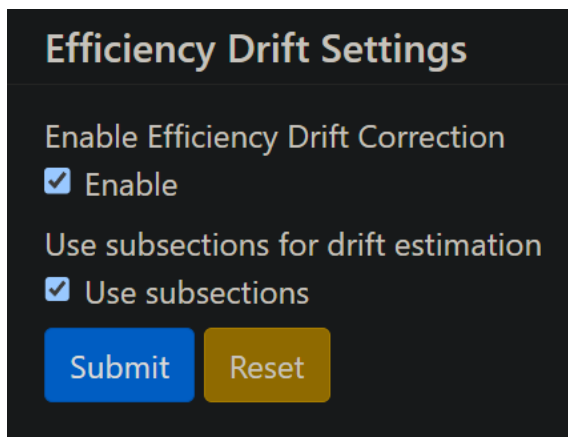


Figure 12 Efficiency drift settings in GUI

correction by introducing the user configured Boolean *useSubsection* = *True*.

useSubsections is enabled in the GUI as a checkbox as seen in figure 12 by the user along with the toggle on and off for the efficiency drift correction. Different subsections were experimented with settling on the final implementation of 12 subsections in a 3 x 4 grid. This once more achieved a balance between complexity, efficiency and performance where a greater number of divisions would not be worth the performance bonus for the computational cost. This value is hard coded to reduce GUI complexity for the user.

Algorithm 5 *efficiency_drift_step*

```
PROCEDURE efficiency_drift_step(app_state):
BEGIN
    image_sums = SUM(image) FOR image IN app_state['current_images']
    IF useSubsections THEN:
        Divide images into 3x4 grid of subsection
        FOR each subsection do
            Compute sum of pixel values across frames
            Fit linear regression
        END FOR
    ELSE:
        Fit linear regression (frame, pixel sum)
        avg_gradient ← Mean of all subsection gradients
        avg_intercept ← Mean of all subsection intercepts
    ENF IF
    corrected_images = []
    FOR i from 0 to N-1 do
        IF i ∈ anomalies THEN
            APPEND image TO corrected_images
            CONTINUE
        END IF
        image_sum ← SUM(pixels values in image i)
```

```

drift_value ← avg_slope × i + avg_intercept
corrected_sum ← image_sum - drift_value + avg_intercept
correction_factor ← corrected_sum / image_sum
corrected_image ← image_i × correction_factor
APPEND corrected_image TO corrected_images
END FOR

app_state['current_images'] ← corrected_images
END

```

The ability to toggle the subsection method requires visualisations that demonstrate to the user the effect on their data the correction will have to inform their processing decisions. Figure 13 is an example dataset visualisation using the matplotlib python library in the final application.

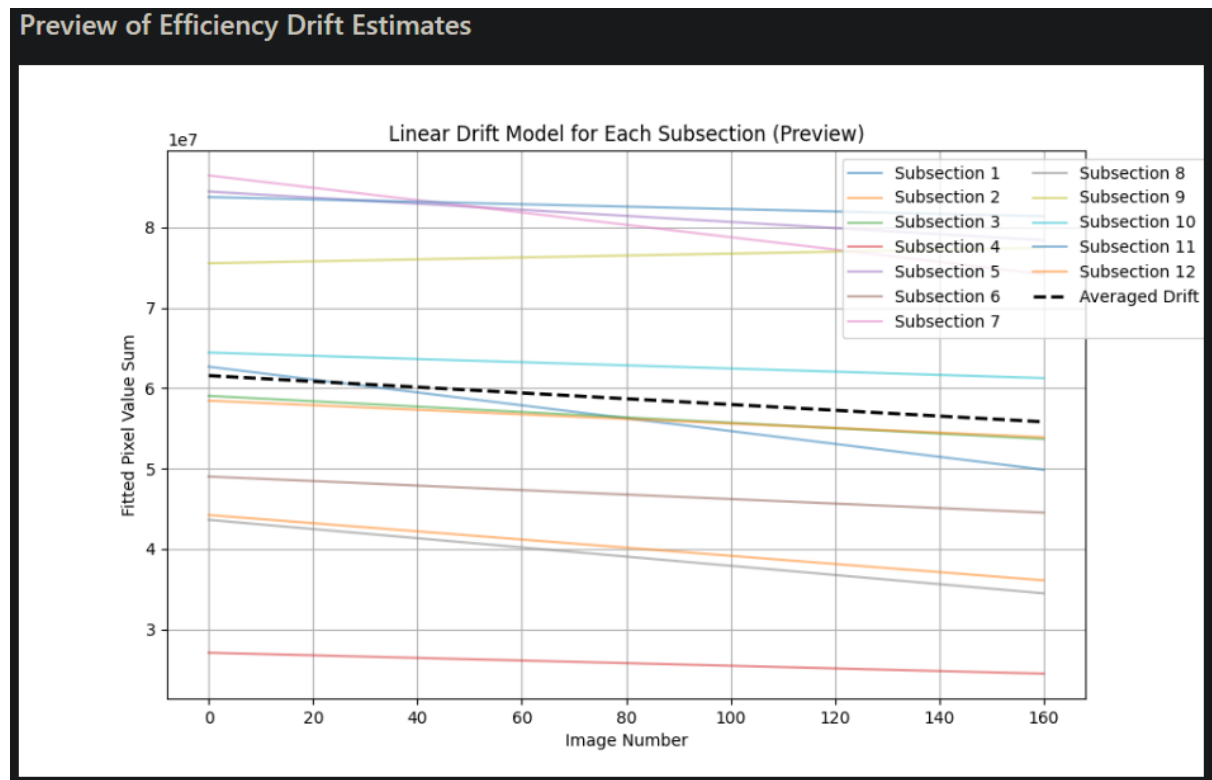


Figure 13 A preview of each drift model calculated independently for each subsection of the image series. The dashed line shows the final model averaged across parameters from each of the 12 subsections.

The subsection computation was slow for large datasets. This was solved by using optimised NumPy operations and parallelised loops to improve processing time to within the required time limit (<5 minutes for a standard dataset of ~200 images).⁹⁰

4.4 Outputs and GUI

The GUI was the final phase of development using PyWebIO's output functions `put_markdown` (to output markdown text to the web browser) and `put_image` (to output an image to the web browser). Further functions `put_input` (to output persistent input field for numerical parameters) and `put_checkbox` (which outputs toggleable checkboxes to the web browser for user configurable Booleans) were utilised for user-configuration in the UI. All these widgets were 'pinned' to the page using PyWebIO's pin module, creating persistent UI elements that maintain their state throughout the process of the workflow [33]. The interface follows a 7-step navigation bar seen in Figure 14 with a progress bar as described in design section 3.

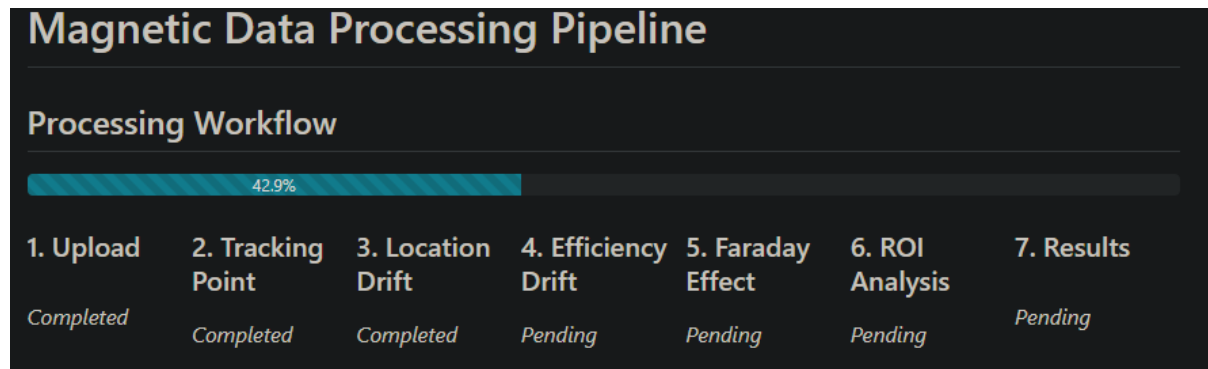


Figure 14 the progress bar implementation in the final GUI during a runtime session when the user is at step 4 of the pipeline.

The progress bar is updated and controlled via the `app_state` dictionary under the progress key. Custom HTML enabled the draggable rectangle for fixed point and ROI selection. The user feedback and interactivity requirements were met using PyWebIO for success and error messages as well as graphs being rendered using Matplotlib and converted to PNGs via an implemented `plot_to_png` function.

5 Testing and Evaluation

5.1 Testing overview

This chapter evaluates the Kerr Microscopy image processing pipeline in its final web application format, assessing its performance against the design requirements outlined in section 3. The testing methodology follows an iterative agile-inspired approach that validates each module of the pipeline's functionality independently on real Kerr Microscopy datasets. The experimental setup is outlined along with testing results and an evaluation of said results.

The pipeline was tested on a desktop system with an AMD Ryzen 7 2700 Processor (3.2 Ghz), 16 Gigabytes (GB) of random-access memory (RAM), windows 11, running python 3.8. This setup reflects a typical researchers workstation in 2025 which allows the portability of the application be tested, with a reasonable but not extensive set of hardware components. Three different Kerr Microscopy datasets were used, each comprising 160 images (or frames) at 960×642 resolution, sourced from experiments on magnetic thin films such as 250 nm thickness of BiFeO₃ (a ferroelectric material often studied in Kerr Microscopy). These datasets include metadata to be extracted enabling comprehensive testing of all pipeline components including OCR functionality.

The testing environment uses PyWebIO's lightweight server to host the web application to be accessed through a Google Chrome browser running on version 35.0.7049.115. This setup ensures compatibility with major browsers which was a requirement established in section 3. Dependencies are installed via a provided *prerequisites.txt* document, meeting the full objective of ensuring the pipeline is portable. A variety of tools will be used to enable testing including python's time module, python's Matplotlib library for graphs and NumPy for computing performance metrics. Each module is tested sequentially following the modular architecture in section 3.5 where tests will be repeated 3 times per dataset to ensure consistency (utilising iteration in testing as part of the agile development cycle).

5.2 Testing

The testing methodology was designed with the performance requirements and user stories from section 3 in mind, using a combination of automated metrics, manual inspections, and user-driven validation. Each module is assessed for performance, efficiency and usability with the following specialised tests in sequential order:

Pre-Processing

Pre-processing the data involves multiple steps, the first being user input for image data and manual identification of anomalous frame indices via the comma-separated textbox. Correct parsing and storage of the image and anomaly data is verified by observing the contents of the `app_state` dictionary throughout the application runtime. The efficiency of pre-processing is measured by the time to upload and decode all 160 frames of the image datasets. OCR is the final stage of pre-processing where the accuracy for timestamp and field data extracted using Pytesseract is compared to true values in a labelled external metadata document.

Drift Correction Algorithms

The performance of the template matching algorithm is measured by computing the standard deviation of residual drift after template matching, targeting <1 pixel. The displacements calculated will be graphed using Matplotlib plots. The test will involve user-configurable parameters with a minimum displacement threshold of 0.5 pixels and a maximum step size of 3 pixels. The effectiveness of the interactive fixed-point selection in the GUI will be evaluated also. The pixel sums of detector drift compensated images in series is plotted before and after correction to confirm the trend removal. The implementation is tested using the *useSubsections* user-configurable parameter. The efficiency of this algorithm is measured by recording the processing time for all 160 frames with and without the use of subsections.

Faraday Effect Separation

CV reduction is calculated as specified in section 3 on the image data after the effect has been removed. This helps evaluate the performance of the model and visualisation of the pixel sums data confirms trends are removed by identifying flat regions at the beginning and end of the data. The usability of this separation technique in the webapp is also evaluated considering the visualisation using graphs of the estimated / corrected effect for user decision-making.

ROI Analysis and Outputs

The performance accuracy of outputs of is validated by analysing the coercivity calculations from hysteresis curves and comparing the values to expected coercivity values from literature e.g., $\sim 25 - 35$ mT for similar materials). The usability of ROI selection and playback of MP4 video

outputs is tested within the browser for functionality. Efficiency of outputs measures time to generate the CSV and videos from the entire 160 frame dataset.

End-to-End Pipeline

Timing the entire pipeline runtime for a standard 160 frame dataset to determine the efficiency of the entire application. The duration required to process the images after parameters are selected between the output page is recorded targeting <3 minutes. The memory usage of the application will also be monitored by using python's psutil library. This test ensures the program is both portable to similar systems and scalable for larger quantities of image data. The responsiveness of the overall UI will be assessed as well its informativeness by providing loading indicators for every process, error handling and success messages.

5.3 Results

5.3.1 Pre-Processing

The pipeline was tested on three separate datasets of Kerr microscopy image data all obtained from the same microscope environment specified in the overview of this section. Pytesseract achieved a 95% accuracy for timestamps extraction where timestamps were able to be parsed with errors primarily due to character mis categorisation. Timestamp data extracted from within images was smoothed to reduce error when graphing for hysteresis curves. Field data was correctly merged with timestamps using Pandas to produce the output CSV correctly with scale for each image also extracted correctly. Handling of user-specified anomalous frames were correctly excluded from processing, verified by observing the content of `app_state['anomalies']` during runtime. The comma-separated textbox introduced no identifiable errors as its use was intuitive for users with no parsing errors across all 9 inputs (3 trials for each of 3 datasets). Overall image uploading and decoding of 160 frames took an exceptional average of 1 second (see table 2).

5.3.2 Spatial Drift Correction

Template matching achieved a residual standard deviation of 0.67 pixels for dataset 1, 0.79 for dataset 2 and 0.69 for dataset 3. All these values are below the threshold of <1 pixel outlined in the design requirements. The average residual standard deviation in the data across all tests and datasets was 0.72 meeting the objective criteria and addressing satisfactorily the user story requiring effective spatial drift correction. Figure 15, demonstrating how the improvements that

were made to the template matching algorithm resulted in significantly improved performance.

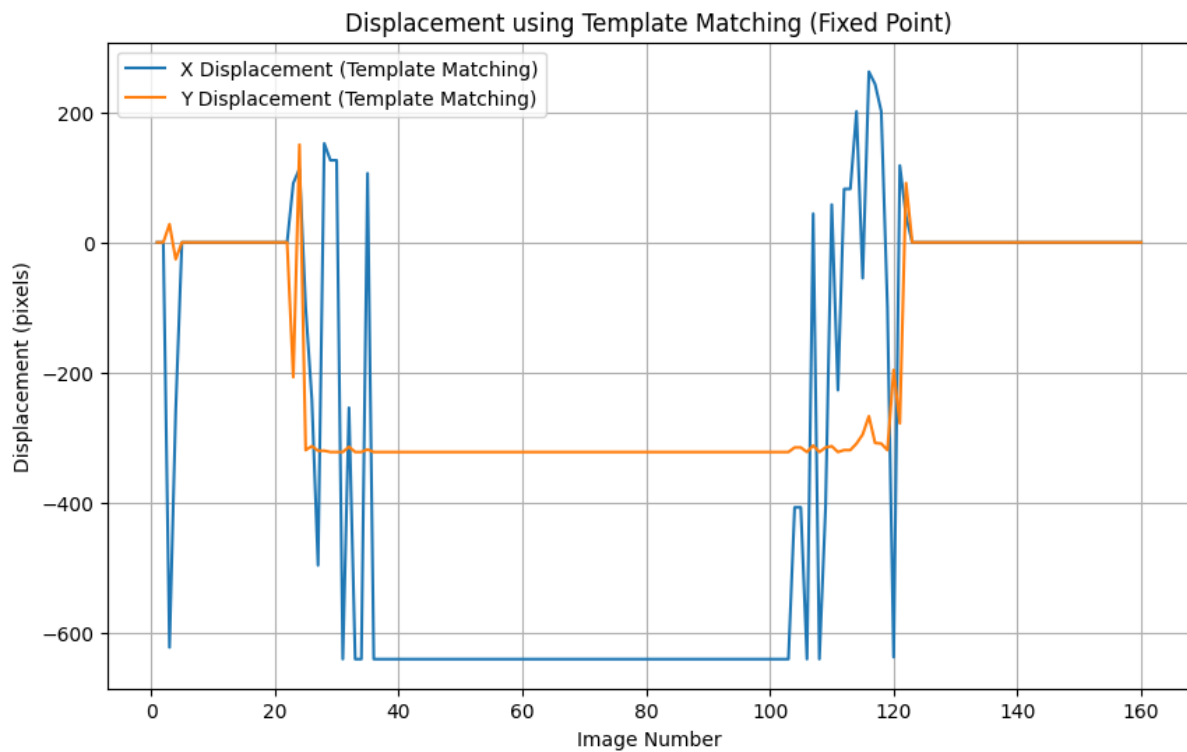


Figure 15 Early plot mapping observed displacement against image index identified by the initial implementation of the template matching algorithm. There are clear performance issues that have been addressed by the improved implementation.

The initial template matching algorithm had issues of identifying and carrying over extreme displacements that did not reflect the data. These issues were addressed in the final implementation through the methods described in section 3 and 4. Comparatively the equivalent early draft functionality performance of RANSAC-based affine transformation saw a significantly different trend in the displacement graph, where the displacement bounced around each image with a high variance. RANSAC's best performance was able to achieve 1.4-pixel standard deviation in residual drift on the Kerr image datasets. KLT point tracking was determined as unsuitable for this application as its approach doesn't typically rely on a fixed point and achieved suboptimal results (2.9-pixel standard deviation) for the datasets. The optimal minimum threshold was identified through trial-and-error testing as 0.5 pixels and maximum step size of 1.5 pixels which prevented large drift anomalies as these shouldn't be present and don't reflect the nature of microscopy image data. Adjusting the template size had a varying effect on accuracy and performance with a smaller template size over the fixed-point region more frequently improving performance. The interactive fixed-point assignment using a draggable shape on a canvas of the first image proved intuitive with user testing resulting in users completing the allocation in <10 seconds across all of 5 test sessions. The custom HTML and JavaScript enhanced the interface and provided real time interaction and feedback for the user, lowering the barrier of entry for researchers looking to user the tool (addressing user stories requiring good usability). Processing all 160 frames took an average of around 4 seconds as observed in table 2. This contributed minimally to the interactive pipeline time and maintained efficiency.

5.3.3 Detector Drift Correction

Figure 7 shows subsection-specific $S_k(t)$ detector drift models and the averaged global mode, confirming linear trend removal and previewing to the user the effect of this algorithm (meeting the informative UI criteria and user story). Using only the first and last images minimised initial overfitting in the model which was an improvement over the initial tests to the final implementation. The subsection toggle improved performance generally and the use of plots allowed users to make an informed decision on whether this parameter was optimal. Global correction took ~3.8 seconds however subsection-based calculations resulted in a 6 second increase to the efficiency of the model. Overall, the use of checkboxes on the UI for the drift correction parameter step was intuitive with supporting use of matplotlib plots enabling flexibility in user parameter selection.

5.3.4 Faraday Effect Separation

The Faraday effect separation uses 7 images for initial and final fits. These values resulted in a performance increase over initial iterations with 12 initial and final values. The piecewise linear model-maintained performance across datasets aligning with the design assumption of global Faraday trends. When graphing pixel sums for each image and correcting for detector efficiency, it was assumed that any residing gradient before the initial magnetic domain contrast effects were the result of a mixture of noise and Faraday effect.

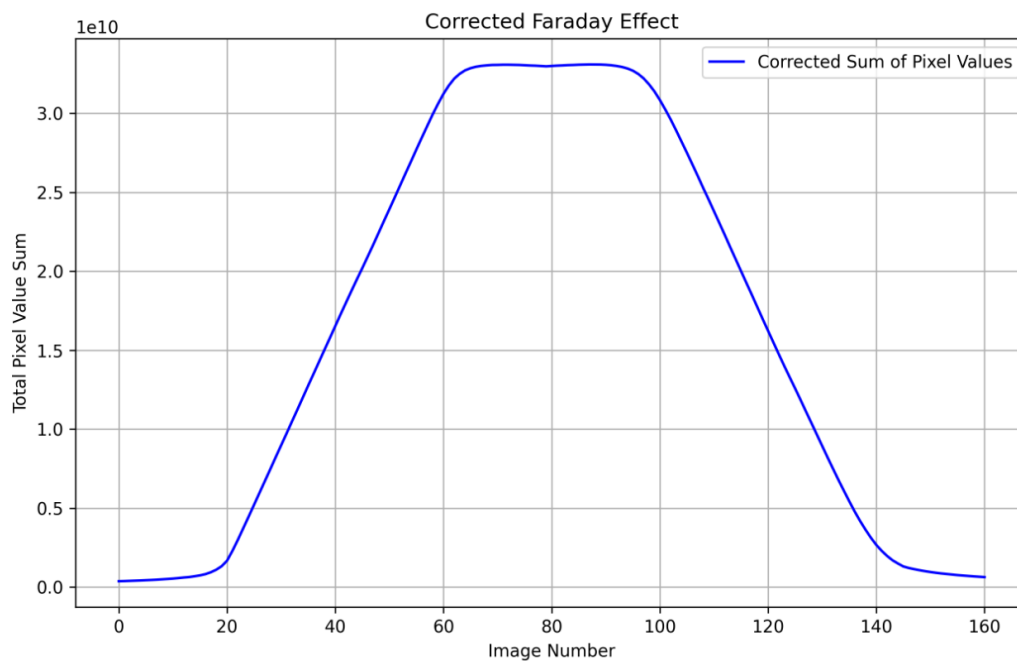


Figure 16 Pixel-Sum curve corrected for Faraday.

Observing the same graph after the separation of this effect reveals a primarily flat region at the beginning and end of the graph, indicating a reduction in variation across pixel sums because of the correction. Processing took ~12 seconds with negligible differences between changing average parameter settings. This accounted most of the processing time during the parameter election part of the pipeline.

5.3.5 ROI Analysis and Outputs

Hysteresis curves, generated from ROI pixel sums and field levels, yielded high accuracy coercivity values consistent with literature for similar magnetic thin films. Figure 17 shows a coercivity graph for one of the test datasets.

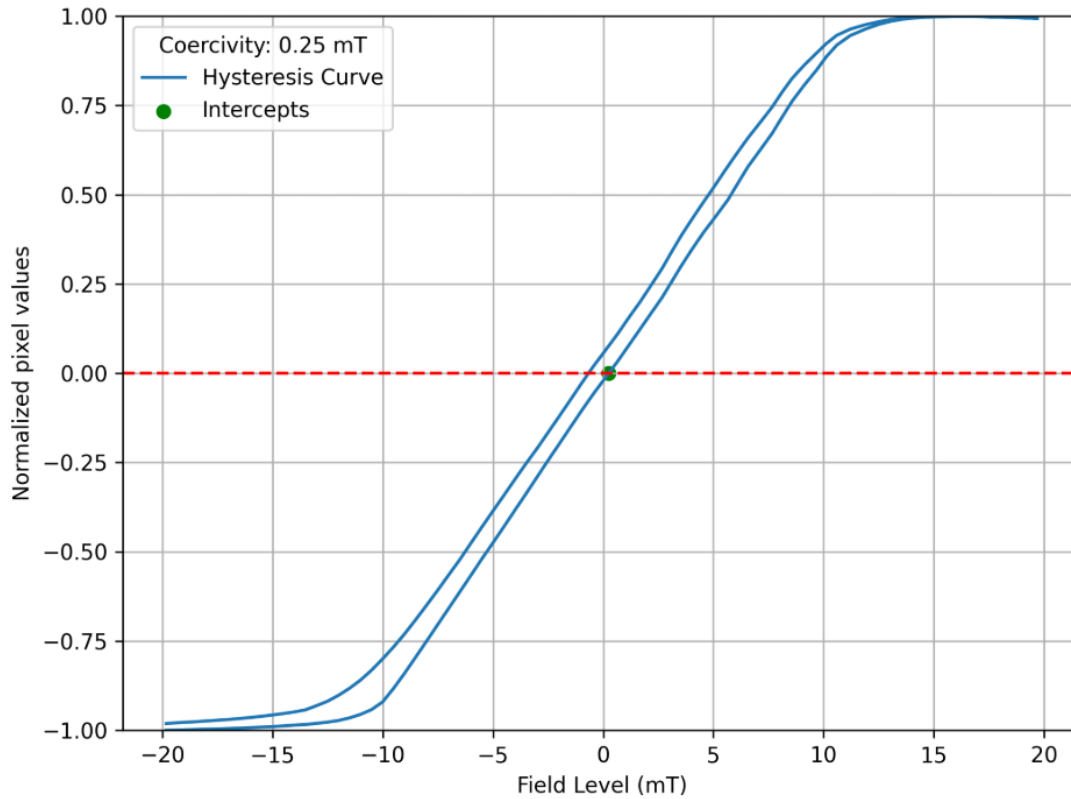


Figure 17 Final hysteresis graph for dataset 1 of the real Kerr image datasets with coercivity calculations.

CSV exports matched expected metadata for values that were parsed correctly and were verified manually. Issues of timestamp parsing prevented consistent metadata extraction. Generating output hysteresis curves and CSV files took 2.1 seconds while video creation took 14.5 seconds for 160 frames.

5.3.6 End-to-End Pipeline

Table 1

Component	Duration (s)		
	Dataset 1	Dataset 2	Dataset 3
Full Processing time	78.52531	81.12485	83.30184
Image Upload	1.006502	0.972996	0.941997
Fixed Point Detection	0.027668	0.028003	0.027999
Template Matching	3.976518	4.003988	3.967977
Efficiency Drift Correction	3.574851	3.839986	2.999988
Faraday Separation	11.66875	6.423981	3.294992
ROI Analysis	1.210108	1.453	3.111993

The full pipeline processed 160 frames of data including all image corrections in 80 seconds ~ 1.3 minutes. This was considerably below the 3-minute target meeting the requirement for

efficiency in the application. Peak memory usage across all test sessions rose to 1119 MB or about 1.1 GB scaling to ~3GB for 500 frames, meeting the memory management requirement and allowing for scalability. Table 2 shows processing time breakdowns per module, highlighting

Table 2

Dataset	Coercivity Calculated (mT)	Expected ~ (mT)	Error (%)
1	0.2508	0.25	0.319
2	0.3899	0.35	10.233
3	0.3356	0.35	4.291

that the interface minimises user waiting times by processing data at the end of the pipeline. An additional test of 500 frames took ~3 minutes, confirming scalability. NumPy's memory-efficient arrays and the use of in-memory processing ensure performance does not degrade as the dataset scales. Testing was performed primarily on chrome however the application functioned consistently when tested across both Firefox and Edge with no browser specific issues

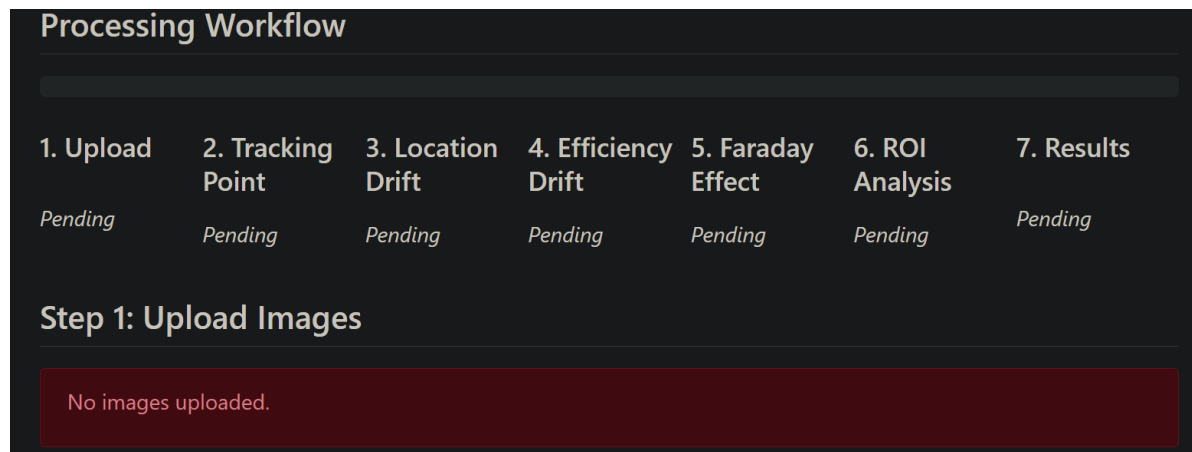


Figure 18 The final workflow progress bar showing progress through the entire pipeline in the application GUI.

demonstrating compatibility. The temporary directory for storing static files to be deployed to the browser during runtime were cleared with no issue between sessions ensuring an out-of-the-box experience. Finally, the overall hysteresis curve where coercivity is calculated can be observed in figure 17. Table 2 documents the average coercivity calculated across testing for each dataset. With an average error of 4.9% largely influenced by the outlying performance on the more variable dataset 2, the extraction of magnetic information from image pixel data demonstrates a successful correlation with actual magnetic properties. The UI remained responsive through all test runs with loading indicators and progress bar updates providing clear feedback at every step. Error handling was tested and caught missing uploads and invalid parameters displaying informative error messages such as in Figure 18.

	Dataset 1	Dataset 2	Dataset 3
Peak Memory Usage (MB)	1110.6	1109.9	1118.9

5.4 Discussion

5.4.1 Evaluation against success criteria

The pipeline met several key requirements including efficiency objectives for processing speed that was recorded as 80 seconds for 160 frames in less than the 3-minute target outlined. The peak memory usage throughout runtime duration was 1.1 GB meeting scalability objectives with compatibility also being achieved across all major web browsers. The portability of the system was achieved further with an out-of-the-box design ensuring static directories are reset at the start of each runtime. The UI was consistently responsive across all tests, with intuitive features like the draggable fixed-point selection and ROI canvas selection. These implementations addressed the user stories for usability and informative UI established in the design section. Coercivity values had a varying error rate that although demonstrated a positive correlation with expected magnetic characteristic could have been improved. An error within field extraction or hysteresis computation is suggested despite the low average error of 4.9% due to the irregular nature of the hysteresis trend specifically requiring smoothing due to variance. The pipelines efficiency stems from optimised NumPy and OpenCV operations that were planned before development began. The agile-based modular design allowed for iterative testing and development which made early implementation more rapid however the dependence on RANSAC methods early in development delayed drift correction improvements. Template matching was chosen as the best option in the absence of labelled training data, but future datasets could utilise machine learning based drift detection with an improved performance. If beginning this project again, more research would have been directed towards drift correction before programming begun. Switching from Streamlit to PyWebIO had its own challenges such as the dependency on custom HTML for interactivity, however this decision enhanced overall usability of the final product and was successful. Overall, the balance between performance and accuracy was partially but detector drift and faraday correction lack some optimisations.

5.4.2 Comparison with existing approaches

Compared to existing approaches for image processing in Kerr microscopy such as MATLAB and ImageJ, the pipeline is significantly faster. ImageJ takes 30 minutes to process 160 frames and lacks spatial drift correction [34]. Comparatively the pipeline is far more efficient and user friendly due to the intuitive UI. MATLAB alternatively offers more similar functionality however is proprietary nature lacks the portability and UI once more of this project. MATLAB-based scripts have a high threshold for entry due to its unique syntax [35]. Template matching outperformed RANSAC (1.4 pixels standard deviation) and KLT (2.9-pixel standard deviation) though recent standard deviation increase in testing warrants further investigation. Template matching having the superior performance was expected based on background research [36].

5.4.3 Discussion of Limitations

The accuracy of OCR text extraction was good when data was parsed >95% although occasionally was limited by timestamps being unable to be parsed. This should be investigated as its suggested that a machine learning based OCR model may optimise performance. Furthermore, the Faraday estimation model struggled with non-linear trends due to the nature of linear regression. I propose the use of polynomial models as a future improvement to enhance performance of this module. Coercivity discrepancies highlight these potential OCR errors in field extraction, as in datasets where OCR parsing is flawless the coercivity calculated is more accurate to multiple degrees. Scalability of the application beyond 500 frames has not

been tested and due to heavy memory dependence is likely to impact performance on low-end systems. The solution if this issue became persistent would be to implement a form of batch processing to reduce the workload on memory at any given time while maintaining performance.

6 Conclusions and Future Work

This project delivered a web-based image processing pipeline for Kerr Microscopy image data to automate the analysis of magnetic materials. Each step of processing was implemented as a modular procedure that enabled sequential processing. It addressed user stories and requirements for efficient implementation of different type of drift detection and correction and magnetic property visualisation and extraction. A summary of how the pipeline was built involved a python implementation using the less recognised PyWebIO framework to develop the pipeline into a front-end GUI to support researchers in magnetic imaging fields. Tested on 3 sets of 160 frames of Kerr image data each 960×642 resolution from iFeO3 thin films using a standard setup for a model researcher workstation (16 GB RAM, windows 11, moderate CPU), the performance met key requirements outlined in the project specification.

Out of the objectives initially identified to make these project successful, key milestones were met with the model processing 160 frames in 80 seconds on average across the 3 different datasets and scaling efficiently to 500 frames in the process. Memory usage peaked at ~ 1.1 Gb which met the computational efficiency objectives and scaled similarly well to 3 GB at 500 frames. The UI saw intuitive implementation, with improvements across iterative design from hard coded template regions to the final interactive canvas implementation. These sections were completed in <10 seconds with video outputs playable in-browser, fulfilling user stories for usability of the UI. Finally, the compatibility and portability of the application is demonstrated by its self-contained nature and compatibility across web browsers allowing the system to operate on as many systems as possible.

Reflecting critically, the choice of template matching for spatial drift correction was constrained by the limitation of unlabelled data. Similarly for identifying specific domains and the rate of expansion, machine learning could possibly have seen application. This, however, would require manual data labelling which introduces significant complexity. It's performance could be improved upon in future iterations as machine learning becomes less data intensive or more data becomes widely available. The Faraday model's assumption of global linearity was flawed for the presence of datasets with non-linear trends and could be improved by introducing polynomial modelling. Design choices were well thought out where possible, switching frameworks from Streamlit to PyWebIO and spatial drift algorithms demonstrated the benefits agile software strategy can have on a workflow. Future work offers substantial opportunities to advance the pipeline. Firstly, literature on machine learning implementations have very recently began developing in applications such as grain size analysis from Kerr microscopy images [37]. Utilisation of a convolutional neural network trained on labelled microscopy data could significantly enhance drift correction and presents good opportunity for a separate project. Different processing modules could be enhanced by machine learning (ML) in different ways, such as drift correction to target sub-pixel accuracy. The use of more diverse datasets would furthermore enhance the development of this system particularly with the use of ML. GPU acceleration could be leveraged more greatly using framework like CUDA (which could be investigated to reduce runtimes). Replacing the piecewise linear Faraday model with a

polynomial approach would better capture non-linear trends within the data. The OCR could be significantly improved by using a more powerful ML (e.g. EAST) based model or a custom-trained Tesseract model. Furthermore, broadening the use case of the application to support other formats of microscopy could help future proof the system by aligning with current trends in materials science.

In conclusion the pipeline successfully achieved automated Kerr Microscopy image processing for a researcher user base with high efficiency and low barrier of entry in terms of GUI and portability. The proposed future work aims to optimise gaps unaddressed by this implementation to utilise advanced techniques for faster more reliable magnetic domain analysis for researchers.

Bibliography

- [1] D. V. C. e. al, “2024 roadmap on magnetic microscopy techniques and their applications in materials science,” *Journal of Physics: Materials*, vol. 7, no. 3, p. 032501, 2024.
- [2] S. Losoi, “Magnetic Anisotropy,” 2015.
- [3] R. S. I. V. Soldatov, “Advanced MOKE magnetometry in wide-field Kerr-microscopy,” *Journal of Applied Physics*, vol. 122, p. 153906, 2017.
- [4] P. a. S. R. a. B. J. a. H. D. Kasiraj, “Magnetic domain imaging with a scanning Kerr effect microscope,” *IEEE Transactions on Magnetics*, vol. 22, no. 5, pp. 837-839, 1986.
- [5] M. G. a. P. Vavassori, “The diffracted magneto-optic Kerr effect: what does it tell you?,” *Journal of Physics: Condensed Matter*, vol. 16, 2004.
- [6] J. McCord, “Progress in magnetic domain observation by advanced magneto-optical microscopy,” *Journal of Physics D: Applied Physics*, vol. 48, no. 33, 2015.
- [7] P. C. a. V. S. a. R. Clarke, “Real-time MOKE microscopy made simple,” 2012.
- [8] H. D. a. N. G. a. P. T. G. Cornean, “The Faraday effect revisited: General theory,” *Journal of Mathematical Physics*, vol. 47, no. 1, p. 013511, 2006.
- [9] R. S. Alex Hubert, *Magnetic Domains: The Analysis of Magnetic Microstructures*, Springer, 1998.
- [10] P. J.-K. J. M. B. P. M. Jovičević-Klug, “Investigation of microstructural attributes of steel surfaces through magneto-optical Kerr effect,” *Journal of Materials Research and Technology*, vol. 11, no. 2238-7854, pp. 1245-1259, 2021.
- [11] F. a. R. W. a. H. A. Schmidt, “Enhancement of magneto-optical domain observation by digital image processing,” *IEEE Transactions on Magnetics*, vol. 21, no. 5, pp. 1596-1598.
- [12] C. RINALDI, “Development of a Kerr microscope for the study of magnetic micro and nanostructures,” 2017.

- [13] "Normalized Differential Kerr Microscopy An Advanced Method for Magnetic Imaging," *physica status solidi*, vol. 171, no. 2, pp. 555-562.
- [14] I. S. J. Zehner, "Advanced, Kerr-microscopy-based MOKE magnetometry for the anisotropy characterisation of magnetic films," *Journal of Magnetism and Magnetic Materials*, 2021.
- [15] S. U. L. U. A. Polewko-Klim, "A method for eliminating Faraday rotation in cryostat windows in longitudinal magneto-optical Kerr effect measurements," *Review of Scientific Instruments*, vol. 85, no. 7, p. 073106, 2014.
- [16] J. D. Rueda Torres, "Construction of a Wide-Field MOKE Microscope for Domain Analysis with CNNs," Universidad de los Andes, 2025.
- [17] "Deep learning and correlative microscopy for quantification of grain orientation in sintered FeNdB-type permanent magnets by domain pattern analysis," *Acta Materialia*, vol. 264, no. 1359-6454, p. 119563, 2024.
- [18] I. V. a. S. R. Soldatov, "Selective sensitivity in Kerr microscopy," *Review of Scientific Instruments*, vol. 88, no. 7, p. 073701, 2017.
- [19] S. Chikazumi, "Physics of ferromagnetism," *Oxford University Press*, vol. 2, no. 978-0-19-956481-1, p. 118, 2009.
- [20] P. Rasaili, N. Sharma and A. Bhattarai, *Comparison of Ferromagnetic Materials: Past Work, Recent Trends, and Applications*, 2022.
- [21] D. M. G. V. M. D. a. B. M. Evans, "Domains and domain walls in multiferroics," *Physical Sciences Reviews*, vol. 5, no. 9, p. 20190067, 2020.
- [22] C. D. G. B. D. Cullity, *Introduction to Magnetic Materials*, 2nd Edition, Wiley-IEEE Press, 2011.
- [23] J. a. H. K. Linder, "Superconducting spintronics with magnetic domain walls," *Phys. Rev. B*, vol. 90, no. 10, pp. 104502 - 104511, 2014.
- [24] Z. Q. a. B. S. D. Qiu, "Surface magneto-optic Kerr effect," *Review of Scientific Instruments*, vol. 71, no. 3, pp. 1243-1255, 2000.
- [25] K. Hovořáková, *Kerr microscopy of magnetic microstructures*, 2022.
- [26] S. Huard, "Polarization of Light," *Wiley-VCH*, no. SBN 0-471-96536-7, p. 348, 1997.
- [27] "A Review of Methods for the Examination of Magnetic Domain Structure," *Materials Characterization*, vol. 39, no. 2, pp. 139-167, 1997.
- [28] W. R. Gonzalez RC, *Digital Image Processing*. Fourth edition, Pearson, 2016.
- [29] K.-h. D. (. S.-l. C. (. Y.-c. S. (. & M.-s. L. (. Xia Ding (丁霞), "Effect of optimal aging treatment on magnetic performance and mechanical properties of sintered Nd-Fe-B permanent magnets," *Journal of Central South University*, vol. 23, p. 515–522, 2016.

- [30] J. a. L. J. a. K. J. Vourvoulakis, "Acceleration of RANSAC algorithm for images with affine transformation," *2016 IEEE International Conference on Imaging Systems and Techniques (IST)*, pp. 60-65, 2016.
- [31] G. { . P. a. K. P. a. M. { . Singh}, "Spliced images detection by using Viola-Jones algorithms method," *Materials Today: Proceedings*, vol. 51, no. 2214-7853, pp. 924-927, 2022.
- [32] M. S. N. a. A. S. Aguado, "Chapter 5 - High-level feature extraction: fixed shape matching," in *Feature Extraction & Image Processing for Computer Vision (Third Edition)*, Oxford, Academic Press, 2012, pp. 217-291.
- [33] W. Wang, "pywebio," 2021. [Online]. Available: <https://pywebio.readthedocs.io/en/dev/pin.html>. [Accessed 27 04 2025].
- [34] T. J. Collins, "ImageJ for Microscopy," *BioTechniques*, vol. 43, 2007.
- [35] E. M. M. S. HY Kueh, "Image analysis for biology," *MBL Physiology Course*, pp. pp.1-52, 2008.
- [36] T. A. Chao ZHANG, "Robust Projective Template Matching," *IEICE TRANSACTIONS on Information*, Vols. E99-D, no. 9, pp. 2341-2350, 2016.
- [37] A. K. C. a. A. J. a. T. G. a. F. T. a. D. G. a. T. B. a. G. Schneider, "Grain size analysis in permanent magnets from Kerr microscopy images using machine learning techniques," *Materials Characterization*, vol. 186, no. 1044-5803, p. 111790, 2022.
- [38] Y. a. C. T. a. L. Y.-F. a. G. J.-m. a. H. S. a. D. F.-w. Shen, "A Direct Current Measurement Method Based on Terbium Gallium Garnet Crystal and a Double Correlation Detection Algorithm," *Sensors*, vol. 19, p. 2997, 2019.

Appendix A – Front End

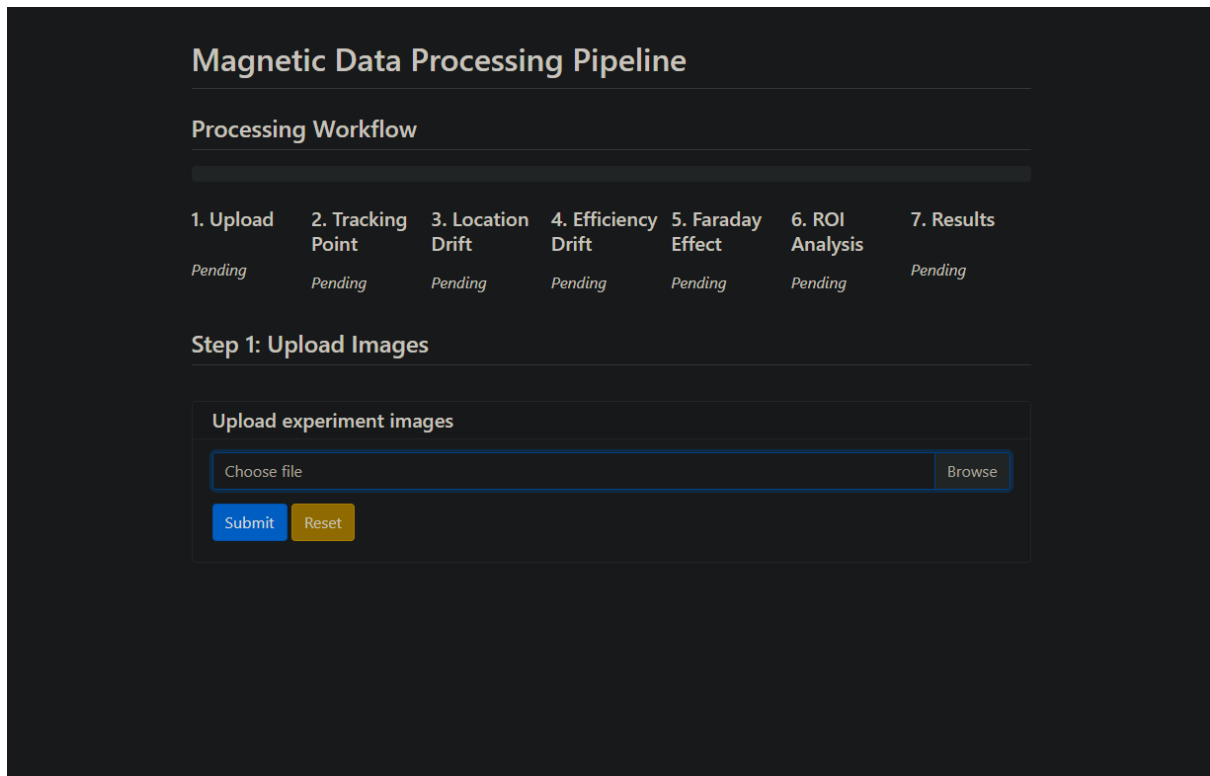


Figure A-1 Front end showing image upload interface step

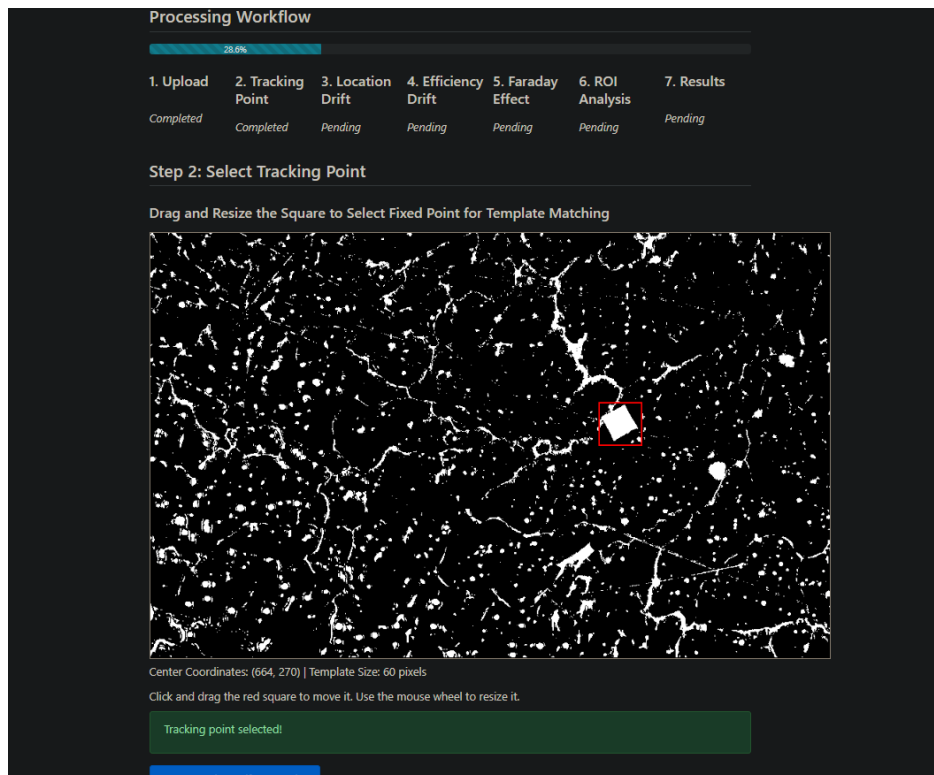


Figure A-2 Front end showing fixed point selection step

Magnetic Data Processing Pipeline

Processing Workflow

1. Upload

2. Tracking Point

3. Location Drift

4. Efficiency Drift

5. Faraday Effect

6. ROI Analysis

7. Results

Pending

Pending

Pending

Pending

Pending

Pending

Pending

Step 1: Upload Images

Enter anomaly indices (comma-separated, e.g., 3)

3

Submit

Reset

Figure A-3 Front end showing anomalous indices textbox step

52

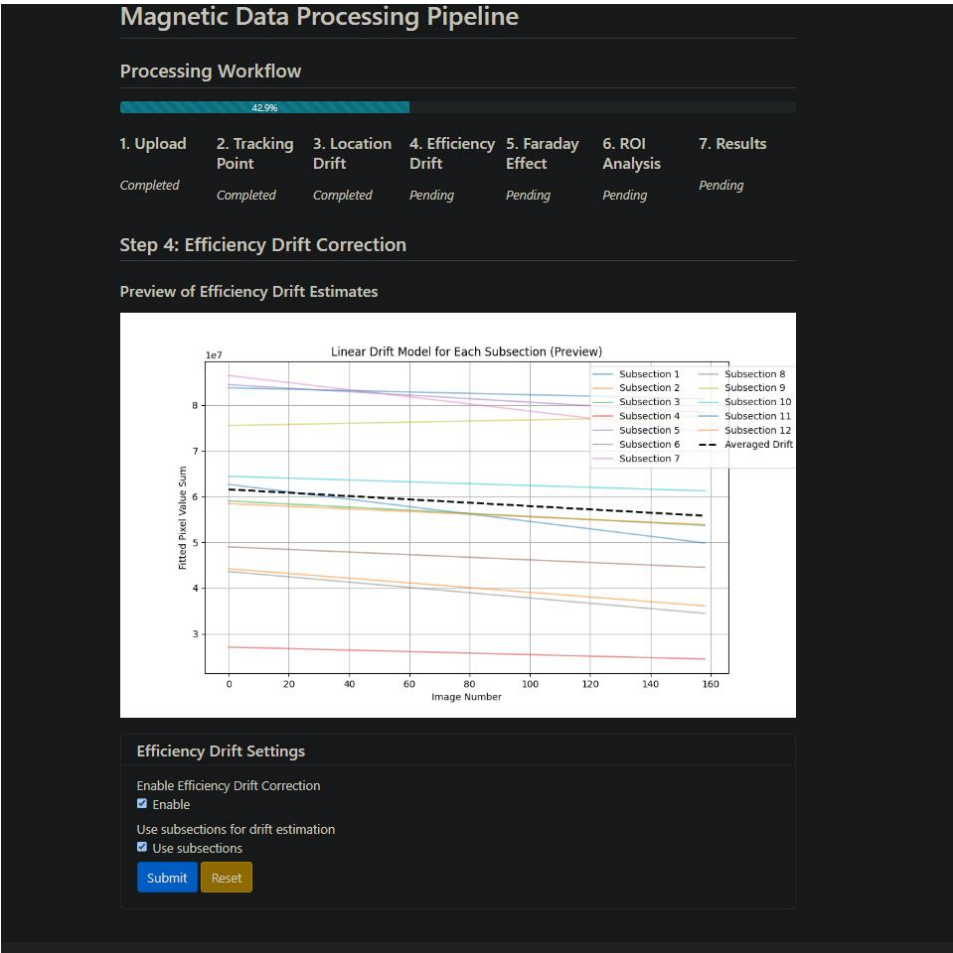


Figure A-4 Front end showing drift subsection parameters step

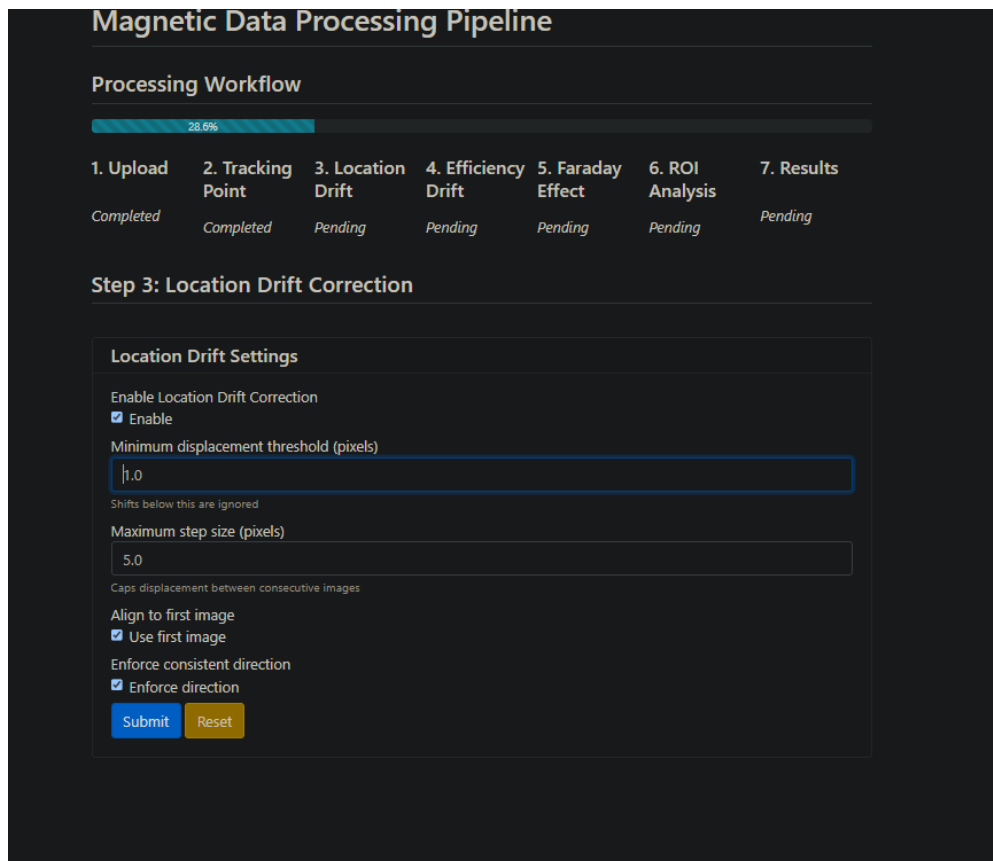


Figure A-5 Front end showing location drift step

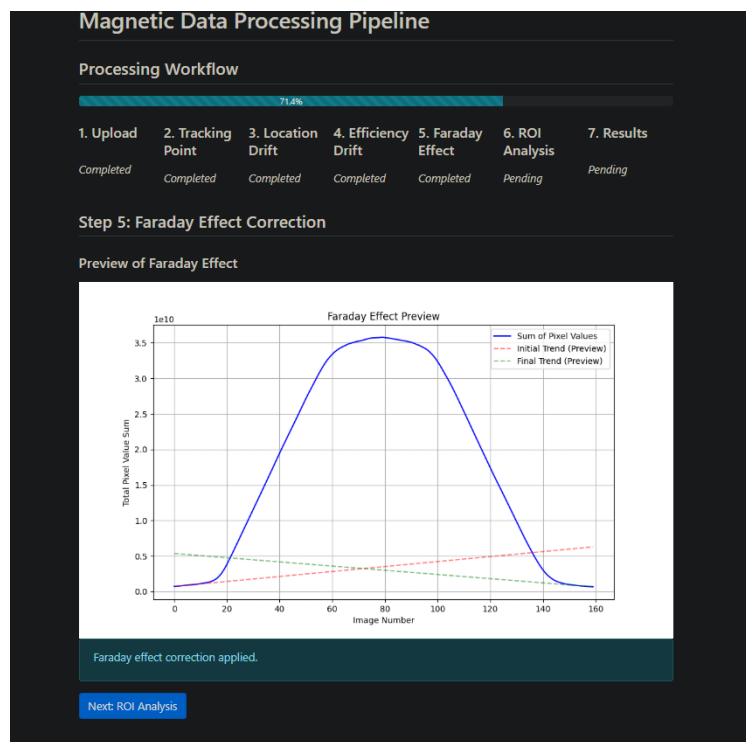


Figure A-6 Front end showing Faraday step

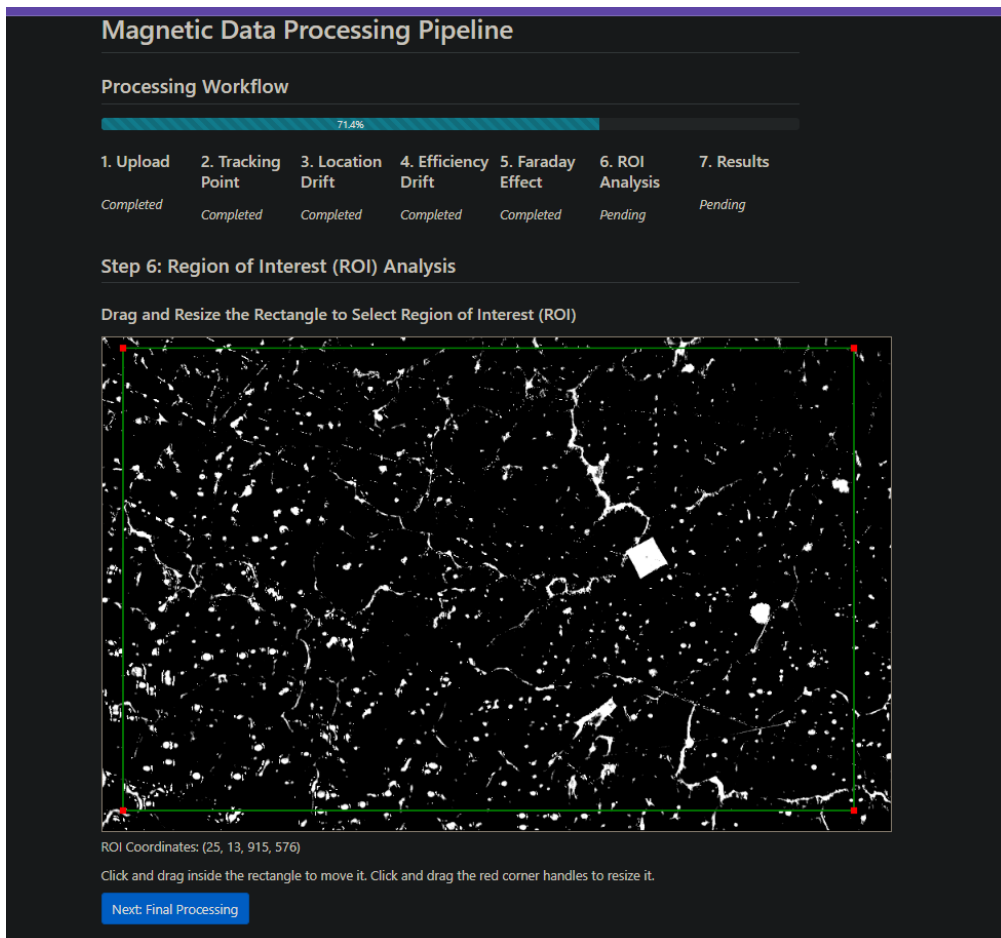


Figure A-7 Front end showing ROI step

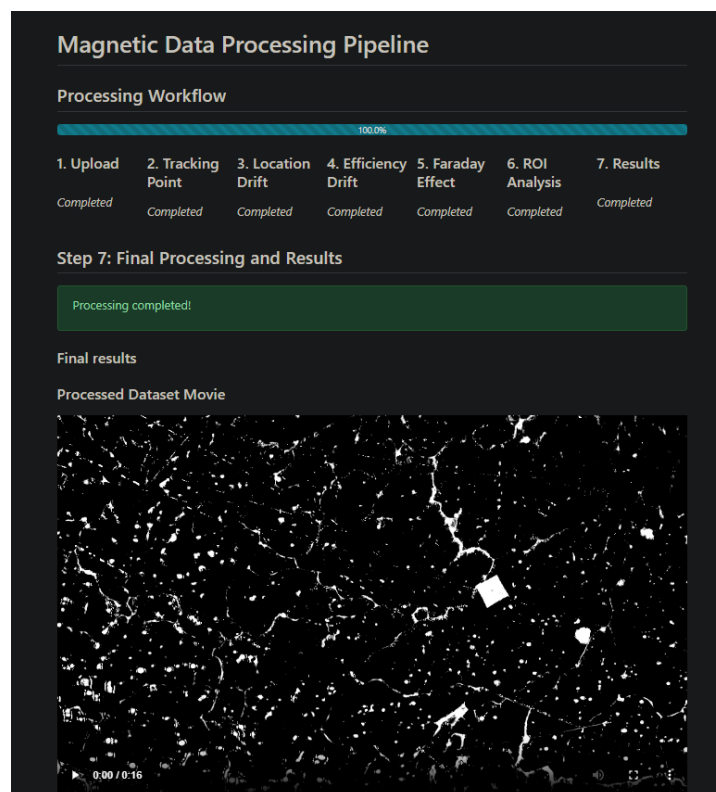


Figure A-8 Front end showing outputs part 1

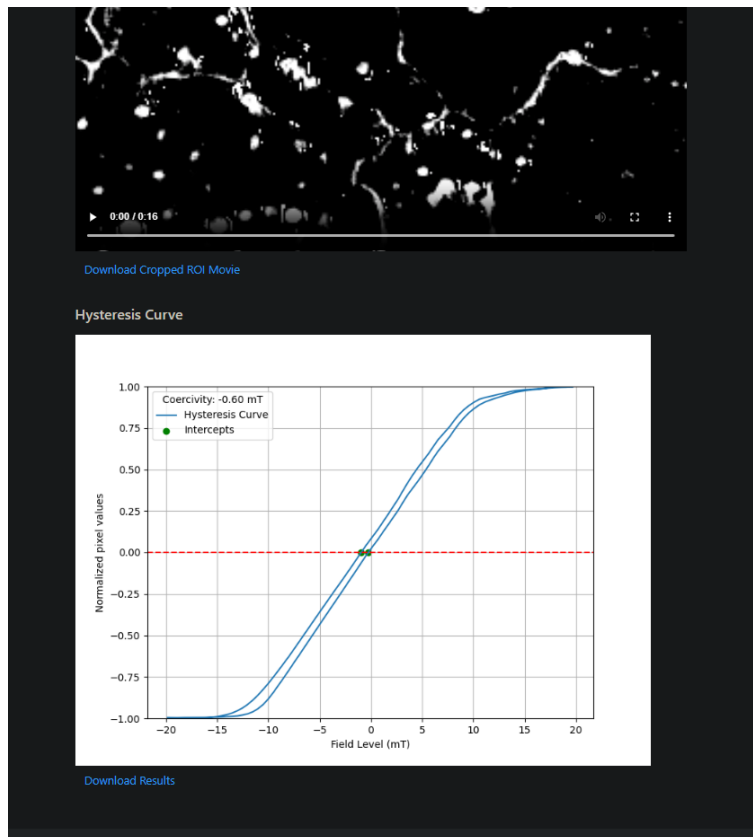


Figure A-9 Front end showing outputs part 2

Appendix B – User Guide

Running the Application

The application was tested on a system with an AMD Ryzen 7 2700, 16 GB RAM, and Windows 11, but it should run on systems with at least 4 GB RAM and an Intel Core 2 Duo.

Here is the full list of dependencies that are necessary to run this program:

- Tesseract-OCR Engine
- Python 3.8
- PyWebIO
- OpenCV
- Pytesseract
- Pillow
- NumPy
- Matplotlib
- Pandas
- Psutil

Here is `prerequisites.txt`:

```
pywebio
opencv-python
pytesseract
pillow
numpy
matplotlib
pandas
psutil
imageio
ffmpeg-python
```

This file can be run using pip to install all prerequisites. This can be done by saving `prerequisites.txt` in the project environment. Ensure python 3.8 is installed by running `python 3.8 --version`. Then run:

```
pip install -r requirements.txt
```

Tesseract-OCR is a system-level dependency and must be installed manually online and added as an environment path.

The application can then be ran using:

```
python CombinedApp.py
```

In the correct directory.

Guide

This app takes a series of individual image uploads. All images must be uploaded in a uniform file format. Images are uploaded individually not as folders. The application expects a series of images uploaded at once. Images should be 960 × 642 resolution greyscale.

The way you upload these images should be to select all the images at once in the file selector and then click upload. It should take images of every type. The images should be indexed by titling them as such: image0000, image0001, image0002 etc. Once images are uploaded you can write the indices of any images you are aware to be anomalous from the dataset.

Most images sets shouldn't have too much location drift but if enabled it is recommended to have either 0 or < 2 pixels of minimum displacement and anything > 5 is likely too much. Optimum values obtained through trial and error were a 0.5 minimum threshold and a 1.5 – 2 pixel maximum.

Next you can correct the detector efficiency either by using the average of 12 image subsections or just by using the estimate calculated across the entire image. You can use the graph at your discretion to determine if this option will be effective for your dataset.

Faraday correction similarly will allow you to average the estimate, however this functionality is different as it averages the gradients to ensure they meet in the middle. If the 2 estimate lines on the graph provided do not line up in the centre it is recommended to enable this average setting.

ROI analysis is recommended for datasets such that we are only interested in one specific region.

On the results page you can view the graph, video and download data extracted from the image dataset.